

```
# ===== main.py =====
# -*- coding: utf-8 -*-
"""
L.Note 桌面版入口
用 pywebview (Edge WebView2 内核) 加载本地编辑器页面，
并通过 js_api 向前端暴露原生文件保存对话框。
"""

import os
import sys
import time
import threading
import base64
import json
import struct
import re
import socket
import shutil
import subprocess
import tempfile
import html as _html
import webview

# 供文件比较窗口读取的待比较数据
pending_compare = None

# 供图片查看器窗口读取的待查看图片
pending_image = None

# 供主编辑器窗口读取的「打开方式」传入的非图片文件
pending_open_file = None

# ---- 单实例 IPC (仅主编辑器进程参与) ----
# 主编辑器进程在 127.0.0.1:IPC_PORT 后台监听；后续「打开方式」启动的进程
# 连接成功并应答 ok 后，把文档路径转发过来并立即退出，不再重复创建主编辑器窗口。
IPC_HOST = "127.0.0.1"
IPC_PORT = 47331

# 主编辑器前端就绪 (frontend_ready 回调) 前收到的「打开方式」文件队列
_runtime_pending_files = []
_runtime_frontend_ready = False

# 版本号 (与 js/app.js 页脚保持一致)
APP_VERSION = "1.0.0"

# 主窗口标题 (软件登记全称，与说明书、源代码文档、界面截图保持同名)
APP_TITLE = "L.Note本地笔记编辑软件"

# 富文档 (块编辑器) 自动存储目录名
RICH_DIRNAME = "L.NoteRich"

def resource_path(rel: str) -> str:
    """兼容开发运行与 PyInstaller 打包后的资源路径。"""
    base = getattr(sys, "_MEIPASS", os.path.dirname(os.path.abspath(__file__)))
    return os.path.join(base, rel)

def _lnote_data_dir() -> str:
    """L.Note 用户数据目录 (%LOCALAPPDATA%/L.Note)，存调试日志与更新结果标记。"""
    return os.path.join(
        os.environ.get("LOCALAPPDATA") or os.path.expanduser("~"), "L.Note"
    )
```

```

def _debug_log(msg: str):
    """写入调试日志（%LOCALAPPDATA%/L.Note/debug.log），用于排查「打开方式」传参问题。"""
    try:
        log_dir = _lnote_data_dir()
        os.makedirs(log_dir, exist_ok=True)
        with open(os.path.join(log_dir, "debug.log"), "a", encoding="utf-8") as f:
            f.write(time.strftime("%Y-%m-%d %H:%M:%S") + " " + msg + "\n")
    except Exception:
        pass

def _parse_clx_pieces(clx: bytes, wd: bytes):
    """解析 CLX（Prc/Pcdt 块）中的 piece table，返回文本片段列表。
    PCD 的 fc 最高位（0x40000000）表示压缩 ANSI（cp1252）piece，
    否则为 UTF-16LE（fc 为字节偏移）。
    """
    out = []
    pos = 0
    n = len(clx)
    while pos < n:
        tag = clx[pos]
        if tag == 0x01: # Prc: 跳过属性
            lcb = struct.unpack_from("<I", clx, pos + 1)[0]
            pos += 5 + lcb
        elif tag == 0x02: # Pcdt: piece table
            lcb = struct.unpack_from("<I", clx, pos + 1)[0]
            pcdt = clx[pos + 5: pos + 5 + lcb]
            plen = len(pcdt)
            if plen < 4:
                break
            ncp = (plen - 4) // 12
            if ncp <= 0:
                break
            cps = struct.unpack_from("<%dI" % (ncp + 1), pcdt, 0)
            for i in range(ncp):
                pcd_off = 4 * (ncp + 1) + i * 8
                fc = struct.unpack_from("<I", pcdt, pcd_off)[0]
                nchars = cps[i + 1] - cps[i]
                if nchars <= 0:
                    continue
                if fc & 0x40000000:
                    fpos = (fc & 0x3FFFFFFF) // 2
                    raw = wd[fpos: fpos + nchars]
                    piece = raw.decode("cp1252", errors="surrogateescape")
                    out.append(_fix_mojibake(piece))
                else:
                    fpos = fc & 0x3FFFFFFF
                    raw = wd[fpos: fpos + nchars * 2]
                    out.append(raw.decode("utf-16-le", errors="replace"))
            break
        else:
            break
    else:
        break

```

```
    return out
def _decode_text_loose(data: bytes) -> str:
    """按常见编码尝试解码文本，全部失败时用替换字符兜底。"""
    for enc in ("utf-8", "utf-16", "gbk", "cp1252"):
        try:
            return data.decode(enc)
        except Exception:
            continue
    return data.decode("utf-8", errors="replace")
def _fix_mojibake(text: str) -> str:
    """还原被 CP1252 误解码的 UTF-8/GBK 文本（mojibake）。
    规则：文本重新按 CP1252 编码回字节（surrogateescape 无损还原 cp1252
    未定义字节），再依次尝试按 UTF-8 / GBK 解码；仅当结果含中日韩字符且
    与原文本不同才采用，避免误伤正常 cp1252 文本。无法还原的残留代理
    字符转为 U+FFFD，保证输出可安全序列化。
    """
    if not text:
        return text
    try:
        raw = text.encode("cp1252", errors="surrogateescape")
    except Exception:
        return text
    for enc in ("utf-8", "gbk"):
        try:
            fixed = raw.decode(enc)
        except Exception:
            continue
        if fixed != text and re.search(r"[\u4e00-\u9fff\u3040-\u30ff\uac00-\ud7af]", fixed):
            return fixed
    if re.search(r"[\udc80-\udcff]", text):
        return re.sub(r"[\udc80-\udcff]", "\ufffd", text)
    return text
def _extract_rtf_text(data: bytes) -> str:
    """简易 RTF 文本提取：剥离控制字与分组，支持 \\uN、\\'xx 转义。
    覆盖 Word/WPS 导出的常见 RTF；不支持的属性控制字一律忽略。
    """
    s = data.decode("latin-1", errors="replace")
    out = []
    i, n = 0, len(s)
    while i < n:
        c = s[i]
        if c == "\\":
            i += 1
            if i < n and s[i] in "{}":
                out.append(s[i])
                i += 1
                continue
            if i < n and s[i] == "'":
                try:
                    out.append(chr(int(s[i + 1:i + 3], 16)))
```

```
        i += 3
    except Exception:
        i += 1
    continue
j = i
while j < n and s[j].isalpha():
    j += 1
word = s[i:j]
k = j
neg = False
if k < n and s[k] == "-":
    neg = True
    k += 1
d = k
while d < n and s[d].isdigit():
    d += 1
param = s[k:d]
i2 = d
if word != "u" and i2 < n and s[i2] == " ":
    i2 += 1
if word == "u" and param:
    v = int(("-" if neg else "") + param)
    if v < 0:
        v += 0x10000
    out.append(chr(v))
    if i2 < n and s[i2] not in "\\{}":
        i2 += 1
    i = i2
elif word in ("par", "line", "cr"):
    out.append("\n")
    i = i2
elif word == "tab":
    out.append("\t")
    i = i2
elif word in ("emspace", "enspace", "qmspace"):
    out.append(" ")
    i = i2
else:
    i = i2
elif c in "{}":
    i += 1
elif c in "\r\n":
    out.append("\n")
    i += 1
else:
    out.append(c)
    i += 1
lines = "".join(out).split("\n")
return "\n".join(ln.strip() for ln in lines).strip("\n")
def _extract_zip_doc_text(path_or_data) -> str:
```

```

"""ZIP 容器: 可能是 .docx/.odt 改名, 读 document.xml 提取文本。
参数可以是文件路径或已读入的字节 (用于 UTF-8 化二进制还原后的解析)。
"""

import io
import zipfile
if isinstance(path_or_data, (bytes, bytearray)):
    zf = zipfile.ZipFile(io.BytesIO(bytes(path_or_data)))
else:
    zf = zipfile.ZipFile(path_or_data)
with zf as z:
    name = None
    for cand in ("word/document.xml", "content.xml"):
        if cand in z.namelist():
            name = cand
            break
    if not name:
        raise ValueError("是 ZIP/OOXML 文档, 但缺少正文流 (word/document.xml)")
    xml = z.read(name).decode("utf-8", errors="replace")
    xml = re.sub(r"<(w:p|text:p)[>]", "\n", xml)
    xml = re.sub(r"<[^>]+>", "", xml)
    return _html.unescape(xml).strip()
def _strip_html_text(s: str) -> str:
    """剥离 HTML 标签与脚本, 保留段落换行。"""
    s = re.sub(r"(?is)<(head|script|style)[^>]*.*?</\1>", " ", s)
    s = re.sub(r"(?i)<br\s*/?>", "\n", s)
    s = re.sub(r"(?i)</(p|div|tr)[^>]*>", "\n", s)
    s = re.sub(r"<[^>]+>", " ", s)
    s = re.sub(r"[\t\u00a0]+", " ", s)
    s = re.sub(r"\n\s*\n+", "\n\n", s)
    return _html.unescape(s).strip()
def _extract_html_text(data: bytes) -> str:
    """HTML 伪装的 .doc: 按页面声明编码解码后剥离标签, 保留段落换行。"""
    head = data[:4096].lower()
    if b"charset=gb2312" in head or b"charset=gbk" in head or b"charset=gb18030" in head:
        s = data.decode("gb18030", errors="replace")
    else:
        s = data.decode("utf-8", errors="replace")
    return _strip_html_text(s)
def _find_html_body(data: bytes) -> int:
    """全文件扫描 HTML 正文起点; HTML 标记可能位于文件深处, 返回 -1 表示未找到。"""
    lower = data.lower()
    pos = -1
    for marker in (b"<!doctype html", b"<html", b"<head", b"<body", b"<meta"):
        idx = lower.find(marker)
        if idx >= 0 and (pos < 0 or idx < pos):
            pos = idx
    return pos
def _looks_like_real_html(text: str) -> bool:
    """粗略判断 HTML 剥离结果是否像真实文档内容 (拦截误判)。"""
    if not text:

```

```

        return False
    bad = sum(1 for ch in text if ord(ch) < 32 and ch not in "\n\r\t")
    if bad > max(8, len(text) * 0.05):
        return False
    if text.count("\ufffd") > max(8, len(text) * 0.1):
        return False
    return True
def _post_process_doc_text(text: str) -> str:
    """对已提取的文档文本统一后处理：还原 mojibake、若含 HTML 则剥离标签。"""
    if not text:
        return text
    text = _fix_mojibake(text)
    if re.search(r"(?i)<!doctype\s+html|<html[\s>]|<head[\s>]|<body[\s>]", text):
        text = _strip_html_text(text)
    return text.strip()
def _extract_ole_doc_text(path_or_data) -> str:
    """用 olefile 从旧版 .doc (Word 97-2003 OLE2 复合文档) 提取纯文本。
    WordDocument 流 FIB (fibRgLw) 中：ccpText 在偏移 96，fcClx 在 120，lcbClx 在 124；
    CLX 位于表流 (1Table/0Table) 的 fcClx 处。
    参数可以是文件路径或已读入的字节（用于 UTF-8 化二进制还原后的解析）。
    """
    import io
    import olefile
    if isinstance(path_or_data, (bytes, bytearray)):
        ole = olefile.OleFileIO(io.BytesIO(bytes(path_or_data)))
    else:
        ole = olefile.OleFileIO(path_or_data)
    try:
        if not ole.exists("WordDocument"):
            raise ValueError("不是有效的 .doc 文件（缺少 WordDocument 流）")
        wd = ole.openstream("WordDocument").read()
        if len(wd) < 128:
            raise ValueError("不是有效的 .doc 文件（WordDocument 流过短）")
        ccp_text = struct.unpack_from("<I", wd, 96)[0]
        if ccp_text <= 0:
            return ""
        fc_clx, lcb_clx = struct.unpack_from("<II", wd, 120)
        if ole.exists("1Table"):
            table_name = "1Table"
        elif ole.exists("0Table"):
            table_name = "0Table"
        else:
            raise ValueError("不是有效的 .doc 文件（缺少表流）")
        table = ole.openstream(table_name).read()
        clx = table[fc_clx:fc_clx + lcb_clx]
        text = "".join(_parse_clx_pieces(clx, wd))
        return text[:ccp_text]
    finally:
        ole.close()
def _try_restore_utf8_wrapped_bytes(data: bytes):

```

"""检测UTF-8 化二进制并还原真实字节。

某些环境保存 .doc 时会把二进制当作 latin-1/cp1252 文本读出，再整体以 UTF-8 编码写入，典型特征：OLE2 头 D0 CF 11 E0 A1 B1 1A E1 变成 C3 90 C3 8F 11 C3 A0 C2 A1 C2 B1 1A C3 A1。此时文件整体是合法 UTF-8，解码后用 latin-1 回编（0x00-0xFF 全部无损）即可还原原始二进制。仅当还原字节匹配已知格式签名（OLE2/RTF/ZIP/HTML）才返回，避免误伤正常 UTF-8 纯文本（中文文本无法以 latin-1 回编，天然被排除）。

"""

```
try:
    text = data.decode("utf-8")
except Exception:
    return None
try:
    raw = text.encode("latin-1")
except Exception:
    return None
if len(raw) < 8:
    return None
if raw[:8] == b"\xD0\xCF\x11\xE0\xA1\xB1\x1A\xE1":
    return raw
stripped = raw.lstrip(b"\xef\xbb\xbf\x00 \t\r\n")
if (
    stripped[:5].upper() == b"\\RTF"
    or raw[:4] == b"PK\x03\x04"
    or stripped[:4] == b"PK\x03\x04"
):
    return raw
if _find_html_body(raw) >= 0:
    return raw
return None
def _extract_doc_core(data: bytes) -> str:
    """文档解析核心：data 为原始或已还原的真实字节。"""
    head = data[:8]
    if head == b"\xD0\xCF\x11\xE0\xA1\xB1\x1A\xE1":
        try:
            return _post_process_doc_text(_extract_ole_doc_text(data))
        except Exception:
            pass # 伪 OLE2（签名匹配但结构损坏）：降级走 HTML/文本兜底
    stripped = data.lstrip(b"\xef\xbb\xbf\x00 \t\r\n")
    if stripped[:5].upper() == b"\\RTF":
        return _extract_rtf_text(data)
    if data[:4] == b"PK\x03\x04" or stripped[:4] == b"PK\x03\x04":
        return _extract_zip_doc_text(data)
    idx = _find_html_body(data)
    if idx >= 0:
        text = _extract_html_text(data[idx:])
        if _looks_like_real_html(text):
            return text
    txt = _fix_mojibake(_decode_text_loose(data))
    control = sum(1 for ch in txt if ord(ch) < 9 or 13 < ord(ch) < 32)
```

```

    if txt and control > max(8, len(txt) * 0.05):
        raise ValueError("无法识别的文档格式（已尝试 OLE2/RTF/HTML/ZIP/文本解析）")
    return txt.strip()
def _extract_doc_text(path: str) -> str:
    """从 .doc 提取纯文本：自动识别 OLE2 / RTF / ZIP(OOXML) / HTML / 纯文本。
    旧版 .doc 在 Windows 生态中常见伪装格式（RTF/HTML/文本），Word 能打开，
    因此除了 OLE2 复合文档还做多格式兜底，保证尽量展示内容。
    另处理"UTF-8 化二进制"：二进制被当作 latin-1 文本保存后整体再以
    UTF-8 编码（文件头形如 C3 90 C3 8F ...），需先还原字节再解析。
    """
    with open(path, "rb") as f:
        data = f.read()
    restored = _try_restore_utf8_wrapped_bytes(data)
    if restored is not None:
        data = restored
    return _extract_doc_core(data)
class CmpApi:
    """文件比较窗口的 API：取回主窗口传入的数据，并支持窗口内加载文件。"""
    def __init__(self):
        self._window = None
    def set_window(self, window):
        self._window = window
    def get_data(self):
        return pending_compare
    def pick_file(self):
        if not self._window:
            return None
        result = self._window.create_file_dialog(
            webview.OPEN_DIALOG, file_types=("所有文件 (*.*)",)
        )
        if isinstance(result, (list, tuple)):
            return result[0] if result else None
        if isinstance(result, str):
            return result
        return None
    def read_file(self, path: str, encoding=None):
        return _read_text_file(path, encoding)
class ImvApi:
    """图片查看/编辑窗口的 API：取回图片数据，并提供覆盖保存与另存为。"""
    def __init__(self):
        self._window = None
    def set_window(self, window):
        self._window = window
    def get_image(self):
        """返回待查看/编辑的图片。单图编辑模式下从磁盘读取并转 data URI。"""
        p = pending_image or {}
        src = p.get("src") or ""
        path = p.get("path") or ""
        name = p.get("name") or "图片"
        if not src and path and os.path.isfile(path):

```

```

        mime = _guess_mime(path)
    try:
        with open(path, "rb") as f:
            raw = f.read()
            src = "data:%s;base64,%s" % (mime, base64.b64encode(raw).decode("ascii"))
    except Exception:
        src = ""
    return {
        "src": src,
        "name": name,
        "path": path,
        "mime": _guess_mime(path) if path else "",
    }

def save_image_file(self, path: str, content_b64: str):
    """把编辑结果（base64，含 data：前缀会被自动忽略）覆盖写回磁盘文件。"""
    if not path:
        return {"error": "目标路径为空"}
    try:
        raw = base64.b64decode(content_b64.split(", ", 1)[-1])
        with open(path, "wb") as f:
            f.write(raw)
        return {"ok": True, "path": path}
    except Exception as e:
        return {"error": str(e)}

def close_window(self):
    """关闭图片编辑窗口。"""
    if self._window:
        self._window.destroy()
    return {"ok": True}

def resize_window(self, width: int, height: int):
    """调整窗口尺寸。"""
    if self._window:
        self._window.resize(int(width), int(height))
    return {"ok": True}

def save_image_as(self, content_b64: str, filename: str):
    """弹出「另存为」保存编辑结果。返回保存路径或 None。"""
    if not self._window:
        return {"error": "窗口未就绪"}
    ext = os.path.splitext(filename or "")[1].lower() or ".png"
    file_types = (
        "图片 (*.*)" % ext,
        "所有文件 (*.*)",
    )
    result = self._window.create_file_dialog(
        webview.SAVE_DIALOG,
        save_filename=filename or "image.png",
        file_types=file_types,
    )
    if isinstance(result, (list, tuple)):
        result = result[0] if result else None

```

```

        if not result:
            return None

    try:
        raw = base64.b64decode(content_b64.split(", ", 1)[-1])
        with open(result, "wb") as f:
            f.write(raw)
        return result
    except Exception as e:
        return {"error": str(e)}

def _read_text_file(path: str, encoding=None):
    """读取文本文件并探测编码（供两个 API 类共用）。"""
    with open(path, "rb") as f:
        raw = f.read()
    enc = encoding if encoding else LNoteApi._detect_encoding(raw)
    try:
        content = raw.decode(enc)
    except (UnicodeDecodeError, LookupError):
        content = raw.decode("utf-8", errors="replace")
    return {
        "content": content,
        "encoding": LNoteApi._ui_enc(enc),
        "size": len(raw),
    }

class LNoteApi:
    """暴露给前端 window.pywebview.api 的原生能力。"""
    def __init__(self):
        self._window = None
    def set_window(self, window):
        self._window = window
    def get_pending_open_file(self):
        """返回右键「打开方式」传入、待主编辑器打开的非图片文件。
        返回 {"path": 绝对路径, "name": 文件名}; 无传入文件时返回 None。
        前端在启动完成后调用，用于自动打开用户选中的文档。
        """
        _debug_log("[api] get_pending_open_file -> " + repr(pending_open_file))
        return pending_open_file
    def frontend_ready(self):
        """主编辑器前端初始化完成回调（单实例接力）。
        前端安装 window.__lnoteOpenExternalFiles 后调用本方法：
        1) 告知后端前端已就绪，可以直接 evaluate_js 推送文件；
        2) 把就绪前第二个实例转发来、已入队的文件一次性冲刷到前端打开。
        """
        global _runtime_frontend_ready
        _debug_log("[api] frontend_ready, queued=" + repr(_runtime_pending_files))
        _runtime_frontend_ready = True
        if _runtime_pending_files:
            items = _runtime_pending_files[:]
            _runtime_pending_files[:] = []
            # 与 translate 的推送一致：在 worker 线程里 evaluate_js，
            # 避免在 js_api 调用栈内同步执行 JS 的重入问题。

```

```

        threading.Thread(
            target=_push_open_to_frontend, args=(self, items), daemon=True
        ).start()
    return True

def consume_update_result(self):
    """新版启动后消费「上次自动更新」的结果标记（一次性）。
    更新守护进程在安装结束后写入 update-result.json:
    成功 {"ok": true, "version": "v0.23.2"}; 失败 {"ok": false, "error": ...}。
    前端据此 toast「已更新到 vX」或失败提示；读取后标记文件即被删除，
    无标记时返回 None（普通启动静默跳过）。
    """
    _debug_log("[api] consume_update_result")
    r = _read_update_result()
    if not r:
        return None
    if r["ok"]:
        return {"updated": True, "version": r["version"]}
    return {
        "failed": True,
        "error": r["error"] or "未知错误",
        "version": r["version"],
    }

def debug_log(self, msg):
    """供前端写入调试日志，排查「打开方式」自动打开链路。"""
    _debug_log("[js] " + str(msg))
    return True

def save_file(self, filename: str, content: str, file_types=None):
    """弹出原生「另存为」对话框，保存文本文件。返回保存路径或 None。
    file_types 可选；为 None 时给「所有文件」单过滤器。"""
    path = self._ask_save_path(filename, file_types)
    if not path:
        return None
    with open(path, "w", encoding="utf-8") as f:
        f.write(content)
    return path

def save_file_binary(self, filename: str, content_b64: str, file_types=None):
    """保存二进制文件（前端传 base64）。返回保存路径或 None。
    file_types 可选；为 None 时给「所有文件」单过滤器。"""
    path = self._ask_save_path(filename, file_types)
    if not path:
        return None
    with open(path, "wb") as f:
        f.write(base64.b64decode(content_b64))
    return path

def read_file_b64(self, path: str):
    """读取任意二进制文件（图片等）为 base64，用于预览内联 / 图片查看器。"""
    if not os.path.isfile(path):
        return {"error": "文件不存在: " + str(path)}
    try:
        with open(path, "rb") as f:

```

```

        data = f.read()
        return {"b64": base64.b64encode(data).decode("ascii"), "mime": _guess_mime(path)}
    except Exception as e:
        return {"error": str(e)}

def copy_image_to_assets(self, dirpath: str, srcpath: str, subdir: str = "assets"):
    """把图片 srcpath 复制到 dirpath/subdir/ 下, 返回最终写入的绝对路径 (正斜杠)。"""
    if not dirpath or not os.path.isdir(dirpath):
        return {"error": "目标目录无效: " + str(dirpath)}
    if not os.path.isfile(srcpath):
        return {"error": "源图片不存在: " + str(srcpath)}
    try:
        dest_dir = os.path.join(dirpath, subdir)
        os.makedirs(dest_dir, exist_ok=True)
        base = os.path.basename(srcpath)
        name, ext = os.path.splitext(base)
        dest = os.path.join(dest_dir, base)
        n = 1
        while os.path.exists(dest):
            dest = os.path.join(dest_dir, "%s_%d%s" % (name, n, ext))
            n += 1
        import shutil
        shutil.copyfile(srcpath, dest)
        rel = (subdir + "/" + os.path.basename(dest)).replace("\\", "/")
        return {"path": dest.replace("\\", "/"), "rel": rel}
    except Exception as e:
        return {"error": str(e)}

def save_image_binary(self, dirpath: str, filename: str, content_b64: str, subdir: str = "assets"):
    """把 base64 图片保存到 dirpath/subdir/ 下, 返回写入的绝对路径 (正斜杠)。"""
    if not dirpath or not os.path.isdir(dirpath):
        return {"error": "目标目录无效: " + str(dirpath)}
    try:
        dest_dir = os.path.join(dirpath, subdir)
        os.makedirs(dest_dir, exist_ok=True)
        dest = os.path.join(dest_dir, filename)
        base, ext = os.path.splitext(filename)
        n = 1
        while os.path.exists(dest):
            dest = os.path.join(dest_dir, "%s_%d%s" % (base, n, ext))
            n += 1
        with open(dest, "wb") as f:
            f.write(base64.b64decode(content_b64))
        rel = (subdir + "/" + os.path.basename(dest)).replace("\\", "/")
        return {"path": dest.replace("\\", "/"), "rel": rel}
    except Exception as e:
        return {"error": str(e)}

def save_file_encoded(self, filename: str, content: str, encoding: str = "utf-8", file_types=None):
    """以指定编码弹出「另存为」, 返回保存路径或 None。
    file_types 可选; 为 None 时给「所有文件」单过滤器。"""
    path = self._ask_save_path(filename, file_types)
    if not path:

```

```

        return None

    data = self._encode_text(content, encoding)
    with open(path, "wb") as f:
        f.write(data)
    return path

LEGACY_RICH_DIRNAMES = ("Ink" + "pad" + "Rich",)

def get_rich_dir(self):
    """返回富文档（块编辑器）的自动存储目录，首次调用时创建。
    富文档可能内嵌多张 base64 图片，体积远超 localStorage 的 ~5MB 上限，
    因此统一自动落盘到该目录（JSON 文件），避免内容丢失。
    目录名曾用旧品牌命名，首次调用时做一次性迁移（同盘 rename, 0(1)）；
    迁移失败（文件被占用等）时回退到旧目录，保证老用户数据不丢。
    """

    import os
    home = os.path.expanduser("~/")
    app_dir = os.path.dirname(os.path.abspath(__file__))
    current = [
        os.path.join(home, "Documents", RICH_DIRNAME),
        os.path.join(app_dir, RICH_DIRNAME),
    ]
    for base in current:
        if os.path.exists(base):
            return base
    legacy = [
        os.path.join(home, "Documents", name)
        for name in self.LEGACY_RICH_DIRNAMES
    ] + [
        os.path.join(app_dir, name)
        for name in self.LEGACY_RICH_DIRNAMES
    ]
    for old in legacy:
        if not os.path.exists(old):
            continue
        new = os.path.join(os.path.dirname(old), RICH_DIRNAME)
        try:
            os.replace(old, new)
            _debug_log("[api] 旧数据目录已迁移: " + old + " -> " + new)
            return new
        except Exception as exc: # noqa: BLE001
            _debug_log("[api] 旧数据目录迁移失败，继续使用: " + old + " (" + str(exc) + ")")
            return old
    for base in current:
        try:
            os.makedirs(base, exist_ok=True)
            return base
        except Exception:
            continue
    return None

def move_rich_file(self, src: str, dst: str):
    """把富文档的磁盘文件移动到新位置（用于「标题改名 → 文件名跟着变」）。
```

同盘走 `os.replace(0(1))`；跨盘降级为 `copy + remove`。

父目录不存在会自动创建。****目标若已存在则直接拒绝****（避免误覆盖其它富文档）。

返回 `{path: 新绝对路径（正斜杠）}` 或 `{error: ...}`。

"""

if not src or not dst:

 return {"error": "源或目标路径为空"}

if not os.path.isfile(src):

 return {"error": "源文件不存在: " + str(src)}

try:

 parent = os.path.dirname(dst)

 if parent:

 os.makedirs(parent, exist_ok=True)

 # 目标已存在 → 拒绝覆盖（前端 `computeRichFilePath` 已尽量避让，

 # 这里再兜底防同盘/竞态）

 if os.path.abspath(src) == os.path.abspath(dst):

 return {"path": dst.replace("\\", "/")}

 if os.path.exists(dst):

 return {"error": "目标已存在，拒绝覆盖: " + str(dst)}

 try:

 os.replace(src, dst)

 except OSError:

 import shutil

 shutil.copyfile(src, dst)

 os.remove(src)

 return {"path": dst.replace("\\", "/")}

except Exception as e:

 return {"error": str(e)}

def delete_rich_file(self, path: str):

 """删除富文档磁盘文件（用于标题改名时清理旧路径）。

 若文件不存在静默成功。返回 `{"ok": True}` 或 `{"error": ...}`。

 """

 if not path:

 return {"error": "路径为空"}

 try:

 if os.path.isfile(path):

 try:

 os.remove(path)

 except OSError:

 # 跨盘 / 权限 → 视作失败但不抛

 return {"error": "无法删除: " + str(path)}

 return {"ok": True}

 except Exception as e:

 return {"error": str(e)}

def list_rich_orphans(self):

 """列出富文档目录下未被 docs 注册占用的 .json 文件（orphan）。

 返回 `{"files": ["绝对路径1", ...]}`；无目录/无 API 时返回 `{"files": []}`。

 JS 端会在启动时拿这个列表跟 docs 里的 `diskPath` 比对，清理掉孤儿文件。

 """

 if not self._window:

 return {"files": []}

```

try:
    base = self.get_rich_dir() if hasattr(self, 'get_rich_dir') else None
except Exception:
    base = None
if not base:
    return {"files": []}
try:
    if not os.path.isdir(base):
        return {"files": []}
    files = []
    for name in os.listdir(base):
        if not name.lower().endswith('.json'):
            continue
        full = os.path.join(base, name).replace("\\", "/")
        # 排除旧版本文件格式 "rich_<id>.json" 之外的「特殊前缀」文件
        # —— 当前实现全部 .json 都视为可能 orphan, 由 JS 决定
        files.append(full)
    return {"files": files}
except Exception:
    return {"files": []}

def cleanup_rich_orphans(self, occupied_paths=None):
    """清理富文档目录下「不在 JS 传过来的占用列表里」的 .json orphan 文件。
    v0.17 标题跟随功能有竞态 bug, 会在改标题过程中残留多份同内容副本。
    启动时调用这个函数可一次清理掉它们。
    安全策略: 只删「内容长得像富文档 + JS 端不认」的文件,
    避免误删用户手动放进该目录的其它 .json。
    返回 {deleted: [...], skipped: [...]}.
    """
    base = self.get_rich_dir() if hasattr(self, 'get_rich_dir') else None
    if not base or not os.path.isdir(base):
        return {"deleted": [], "skipped": []}
    occupied = set((p or '').replace("\\", "/") for p in (occupied_paths or []) if p)
    # 把 src/dst 都规范化, 便于比较
    normalized_occupied = set(os.path.normpath(p).replace("\\", "/") for p in occupied)
    deleted = []
    skipped = []
    try:
        RICH_TYPES = {'text', 'h1', 'h2', 'h3', 'quote', 'todo', 'code', 'table',
                      'image', 'mermaid', 'math', 'callout', 'hr', 'cols',
                      'ulist', 'olist', 'link'}
        for name in os.listdir(base):
            if not name.lower().endswith('.json'):
                continue
            full = os.path.join(base, name)
            full_norm = os.path.normpath(full).replace("\\", "/")
            if full_norm in normalized_occupied:
                continue
            # 嗅探内容, 只删符合富文档格式的
            try:
                with open(full, 'r', encoding='utf-8', errors='ignore') as f:

```

```

        raw = f.read().strip()
        if not (raw.startswith('[') and raw.endswith(']')):
            skipped.append(name)
            continue
        arr = json.loads(raw)
        if not isinstance(arr, list) or not arr:
            skipped.append(name)
            continue
        first = arr[0]
        if not (isinstance(first, dict)
                and isinstance(first.get('id'), str)
                and isinstance(first.get('type'), str)
                and first.get('type') in RICH_TYPES):
            skipped.append(name)
            continue
    except Exception:
        skipped.append(name)
        continue
    # 通过嗅探 → 安全删除
    try:
        os.remove(full)
        deleted.append(name)
    except Exception:
        skipped.append(name)
    return {"deleted": deleted, "skipped": skipped}
except Exception as e:
    return {"deleted": deleted, "skipped": skipped, "error": str(e)}
# ----- 磁盘文件 API (打开文件夹 / 编码转换 / 文件比较) -----
def pick_folder(self):
    """选择文件夹。返回路径或 None。"""
    if not self._window:
        return None
    result = self._window.create_file_dialog(webview.FOLDER_DIALOG)
    return self._first(result)
def pick_file(self):
    """选择单个文件。返回路径或 None。"""
    if not self._window:
        return None
    result = self._window.create_file_dialog(
        webview.OPEN_DIALOG, file_types=("所有文件 (*.*)",)
    )
    return self._first(result)
def pick_files(self):
    """多选文件。返回路径列表。"""
    if not self._window:
        return []
    result = self._window.create_file_dialog(
        webview.OPEN_DIALOG, allow_multiple=True, file_types=("所有文件 (*.*)",)
    )
    if isinstance(result, (list, tuple)):

```

```
        return [p for p in result if p]
    if isinstance(result, str):
        return [result]
    return []

def open_compare_window(self, text_a: str, name_a: str, text_b: str, name_b: str):
    """在独立原生窗口中打开文件比较（左右分栏、差异高亮）。"""
    global pending_compare
    pending_compare = {
        "a": text_a, "aName": name_a,
        "b": text_b, "bName": name_b,
    }
    try:
        threading.Timer(0.2, self._spawn_compare_window).start()
        return True
    except Exception:
        return False

def _spawn_compare_window(self):
    try:
        cmp_api = CmpApi()
        win = webview.create_window(
            "L.Note 文件比较",
            resource_path("compare.html"),
            js_api=cmp_api,
            width=1180,
            height=780,
            min_size=(760, 460),
        )
        cmp_api.set_window(win)
    except Exception:
        pass

def open_image_viewer_window(self, src: str, name: str = ""):
    """在独立原生窗口中打开图片查看器（可放大缩小、拖拽平移）。
    src 为已解析的图片地址：data: URI 或绝对/相对 file:// 路径；
    前端传入 img.src 即可（与页面内渲染用的是同一份）。
    """
    global pending_image
    if not src:
        return False
    pending_image = {"src": src, "name": name or "图片"}
    try:
        threading.Timer(0.15, self._spawn_image_viewer_window).start()
        return True
    except Exception:
        return False

def _spawn_image_viewer_window(self):
    try:
        imv_api = ImvApi()
        win = webview.create_window(
            "L.Note 图片查看器",
            resource_path("image_viewer.html"),
```

```

        js_api=imv_api,
        width=1000,
        height=760,
        min_size=(520, 420),
        private_mode=False,
    )
    imv_api.set_window(win)
except Exception:
    pass
def _first(self, result):
    if isinstance(result, (list, tuple)):
        return result[0] if result else None
    if isinstance(result, str):
        return result
    return None
def list_dir(self, path: str):
    """列出目录直接子项。返回 [{name, path, isDir, size}] 或 {error}。"""
    out = []
    try:
        entries = os.listdir(path)
    except Exception as e:
        return {"error": str(e)}
    for name in entries:
        full = os.path.join(path, name)
        try:
            is_dir = os.path.isdir(full)
            size = 0 if is_dir else os.path.getsize(full)
        except Exception:
            continue
        out.append({"name": name, "path": full, "isDir": is_dir, "size": size})
    out.sort(key=lambda x: (not x["isDir"], x["name"].lower()))
    return out
def read_text_file(self, path: str, encoding=None):
    """读取文本文件并探测编码。返回 {content, encoding, size}。"""
    return _read_text_file(path, encoding)
def read_doc_text(self, path: str):
    """用 olefile 读取旧版 .doc (Word 97-2003) 的纯文本。
    返回 {"text": 提取文本} 或 {"error": 原因}。
    """
    if not os.path.isfile(path):
        return {"error": "文件不存在: " + str(path)}
    try:
        return {"text": _extract_doc_text(path)}
    except Exception as e:
        return {"error": str(e)}
def write_text_file(self, path: str, content: str, encoding: str = "utf-8"):
    """以指定编码写回磁盘文件。自动创建父目录。返回 True。"""
    import os
    # PDF / Word 只读保护: 拒绝任何对 .pdf/.doc/.docx 路径的文本写入 (返回 False, 不抛错)
    if path.lower().endswith((".pdf", ".doc", ".docx")):
```

```

        return False
    parent = os.path.dirname(path)
    if parent:
        os.makedirs(parent, exist_ok=True)
    data = self._encode_text(content, encoding)
    with open(path, "wb") as f:
        f.write(data)
    return True

def show_in_folder(self, path: str):
    """在系统文件管理器中打开 path 所在文件夹，并选中该文件/文件夹。
    Windows 用 explorer /select（打开所在文件夹并选中目标）；
    macOS 用 open -R（Finder 中显示）；其它平台打开所在目录。
    返回 {"ok": True} 或 {"error": ...}。
    """

    import subprocess
    if not path:
        return {"error": "路径为空"}
    try:
        if sys.platform.startswith("win"):
            subprocess.Popen(["explorer", "/select,", os.path.normpath(path)])
        elif sys.platform == "darwin":
            subprocess.Popen(["open", "-R", path])
        else:
            target = path if os.path.isdir(path) else os.path.dirname(path)
            subprocess.Popen(["xdg-open", target])
        return {"ok": True}
    except Exception as e:
        return {"error": str(e)}

# ----- 在线翻译 -----
def translate(self, text: str, target: str = "auto"):
    """翻译文本（在线，需联网）。立即返回 {"started": True}，不阻塞 UI；
    结果由 worker 线程通过 evaluate_js 调用 window.__lnoteTranslateCb 回调推送。
    target 取 "zh"/"en"/"ja"/"ko" 等，缺省 "auto" 按文本内容自动判断。"""
    if not text or not str(text).strip():
        return {"error": "没有可翻译的内容"}
    text = str(text).strip()
    if len(text) > 1500:
        return {"error": "单次最多翻译 1500 字符"}
    if target == "auto":
        target = "zh" if _looks_chinese(text) else "en"
    def worker():
        result = _do_translate(text, target)
        js = "if (window.__lnoteTranslateCb) window.__lnoteTranslateCb(%s);" % json.dumps(result, ensure_ascii=False)
        try:
            if self._window:
                self._window.evaluate_js(js)
        except Exception:
            pass
    threading.Thread(target=worker, daemon=True).start()
    return {"started": True}

```

```

# ----- AI (BYOK) -----
def ai_get_config(self):
    """读取 AI 配置, 返回脱敏视图 {"ok": True, "config": {...}} (不含明文 Key)。"""
    return {"ok": True, "config": _ai_public_view(_ai_read_config())}
def ai_save_config(self, cfg=None):
    """保存 AI 配置。
    cfg 可含 baseUrl / model / temperature / timeoutSec / maxTokens;
    apiKey 仅在用户输入了新密钥时传入; clearKey=true 表示清除已存密钥。
    未传 apiKey 且 clearKey 非真时保留原密钥。返回脱敏视图。"""
    if not isinstance(cfg, dict):
        cfg = {}
    cur = _ai_read_config()
    for k in ("baseUrl", "model"):
        if isinstance(cfg.get(k), str):
            cur[k] = cfg[k].strip()
    for k in ("temperature", "timeoutSec", "maxTokens"):
        if k in cfg:
            try:
                cur[k] = type(_AI_DEFAULT_CONFIG[k])(cfg[k])
            except (TypeError, ValueError):
                pass
    if cfg.get("clearKey"):
        cur["apiKey"] = ""
    elif isinstance(cfg.get("apiKey"), str) and cfg["apiKey"].strip():
        cur["apiKey"] = cfg["apiKey"].strip()
    _ai_write_config(cur)
    return {"ok": True, "config": _ai_public_view(cur)}
def ai_test(self, cfg=None):
    """测试 AI 连通性。以已保存配置为基础, cfg 可覆盖 baseUrl/apiKey/model
    (便于「未保存先测」)。立即返回 {"started": True}, 结果由 worker 线程通过
    evaluate_js 调用 window.__lnoteAiTestCb 推送。"""
    merged = _ai_read_config()
    if isinstance(cfg, dict):
        for k in ("baseUrl", "model", "apiKey"):
            if isinstance(cfg.get(k), str) and cfg[k].strip():
                merged[k] = cfg[k].strip()
        if cfg.get("timeoutSec") is not None:
            try:
                merged["timeoutSec"] = int(cfg["timeoutSec"])
            except (TypeError, ValueError):
                pass
    def worker():
        result = _do_ai_test(merged)
        js = "if (window.__lnoteAiTestCb) window.__lnoteAiTestCb(%s);" % json.dumps(result, ensure_ascii=False)
        try:
            if self._window:
                self._window.evaluate_js(js)
        except Exception:
            pass
    threading.Thread(target=worker, daemon=True).start()

```

```

        return {"started": True}
def ai_chat(self, payload=None):
    """选中即用助手：向配置的模型发起流式对话。
    payload = {"sessionId": str, "messages": [{"role", "content"}],
               "overrides": {"temperature", "maxTokens", "timeoutSec"}}
    立即返回 {"started": True, "sessionId": str}, 不阻塞 UI;
    增量内容与结束态由 worker 线程通过 evaluate_js 调用
    window.__lnoteAiChatCb 推送:
        {sessionId, type:"delta", delta}
        {sessionId, type:"done"|"stopped"|"error", fullText, error?}
    单任务并发：发起新会话会先取消既有会话。"""
    if not isinstance(payload, dict):
        payload = {}
    session_id = str(payload.get("sessionId") or "").strip()
    if not session_id:
        session_id = "ai-" + str(int(time.time() * 1000))
    clean = _ai_clean_messages(payload.get("messages"))
    if not clean:
        return {"error": "缺少对话内容"}
    cfg = _ai_read_config()
    ov = payload.get("overrides")
    if isinstance(ov, dict):
        for k in ("temperature", "maxTokens", "timeoutSec"):
            if ov.get(k) is not None:
                try:
                    cfg[k] = type(_AI_DEFAULT_CONFIG[k])(ov[k])
                except (TypeError, ValueError):
                    pass
    if not str(cfg.get("baseUrl") or "").strip():
        return {"error": "请先在设置中填写 Base URL"}
    if not str(cfg.get("apiKey") or "").strip():
        return {"error": "请先在设置中填写 API Key"}
    if not str(cfg.get("model") or "").strip():
        return {"error": "请先在设置中填写模型名"}
    ev = _ai_begin_session(session_id)
    window = self._window
    def push(obj):
        js = "if (window.__lnoteAiChatCb) window.__lnoteAiChatCb(%s);" % json.dumps(obj, ensure_ascii=False)
        try:
            if window:
                window.evaluate_js(js)
        except Exception:
            pass
    def worker():
        def on_delta(d):
            if ev.is_set():
                return
            push({"sessionId": session_id, "type": "delta", "delta": d})
        result = _do_ai_chat(cfg, clean, on_delta, ev)
        _ai_end_session(session_id)

```

```

        text = result.get("text") or ""
        if ev.is_set():
            push({"sessionId": session_id, "type": "stopped", "fullText": text})
        elif result.get("error"):
            push({"sessionId": session_id, "type": "error", "error": result["error"], "fullText": text})
        else:
            push({"sessionId": session_id, "type": "done", "fullText": text})
    threading.Thread(target=worker, daemon=True).start()
    return {"started": True, "sessionId": session_id}

def ai_stop(self, session_id=None):
    """停止流式生成。session_id 缺省时停止全部会话（单任务并发场景）。"""
    sid = session_id.strip() if isinstance(session_id, str) and session_id.strip() else None
    _ai_stop_session(sid)
    return {"ok": True}

def ai_diagram(self, payload=None):
    """AI 图表生成：由选中文本或描述生成 flow/mind 的「结构 JSON」。
    payload = {"sessionId": str, "kind": "flow"|"mind", "text": str,
               "overrides": {"temperature", "maxTokens", "timeoutSec"}}
    立即返回 {"started": True, "sessionId"}；结果由 worker 线程通过
    evaluate_js 调用 window.__lnoteAiDiagramCb 推送：
        {sessionId, type:"done", structure, title}
        {sessionId, type:"error", error}
    结构 JSON 不含坐标：坐标一律由前端本地布局器计算（PRD §6「图模型」）。"""
    if not isinstance(payload, dict):
        payload = {}
    kind = str(payload.get("kind") or "").strip()
    if kind not in ("flow", "mind"):
        return {"error": "不支持的图表类型"}
    text = str(payload.get("text") or "").strip()
    if not text:
        return {"error": "请先输入用于生成图表的内容"}
    session_id = str(payload.get("sessionId") or "").strip()
    if not session_id:
        session_id = "aid-" + str(int(time.time() * 1000))
    cfg = _ai_read_config()
    ov = payload.get("overrides")
    if isinstance(ov, dict):
        for k in ("temperature", "maxTokens", "timeoutSec"):
            if ov.get(k) is not None:
                try:
                    cfg[k] = type(_AI_DEFAULT_CONFIG[k])(ov[k])
                except (TypeError, ValueError):
                    pass
    if not str(cfg.get("baseUrl") or "").strip():
        return {"error": "请先在设置中填写 Base URL"}
    if not str(cfg.get("apiKey") or "").strip():
        return {"error": "请先在设置中填写 API Key"}
    if not str(cfg.get("model") or "").strip():
        return {"error": "请先在设置中填写模型名"}
    ev = _ai_begin_session(session_id)

```

```

window = self._window
def push(obj):
    js = "if (window.__lnoteAiDiagramCb) window.__lnoteAiDiagramCb(%s);" % json.dumps(
        obj, ensure_ascii=False
    )
    try:
        if window:
            window.evaluate_js(js)
    except Exception:
        pass
def worker():
    result = _do_ai_diagram(cfg, kind, text, ev)
    _ai_end_session(session_id)
    if ev.is_set():
        return # 已取消: 不回推, 前端已丢弃该会话
    if result.get("error"):
        push({"sessionId": session_id, "type": "error", "error": result["error"]})
    else:
        push({"sessionId": session_id, "type": "done",
            "structure": result.get("structure"), "title": result.get("title") or ""})
    threading.Thread(target=worker, daemon=True).start()
    return {"started": True, "sessionId": session_id}
# ----- 关于 / 检查更新 -----
def get_version(self):
    """返回当前应用版本号（与前端页脚保持一致）。"""
    return {"version": APP_VERSION}
def open_external(self, url: str):
    """在系统默认浏览器中打开外部链接（更新日志 / 仓库页 / 下载页）。"""
    import webbrowser
    try:
        webbrowser.open(url)
        return {"ok": True}
    except Exception as e:
        return {"error": str(e)}
def check_update(self):
    """检查最新版本（需联网）。优先 Gitee Releases API（国内直连且始终发布 Release），
    失败回退 GitHub Releases API（仅发布过部分 Release，可能返回旧版）。
    返回 {"ok": True, "latest": "vX.Y.Z", "url": "发布页", "source": "gitee"|"github",
    "current": 当前版本, "update_available": bool}; 异常返回 {"error": ...}。"""
    import urllib.request
    def fetch_latest():
        sources = (
            ("gitee", "https://gitee.com/api/v5/repos/x_xiansheng/l.-note/releases/latest"),
            ("github", "https://api.github.com/repos/stutasliu/LNote/releases/latest"),
        )
        fallback_pages = {
            "gitee": "https://gitee.com/x_xiansheng/l.-note/releases",
            "github": "https://github.com/stutasliu/LNote/releases",
        }
        last = None

```

```

    for name, url in sources:
        try:
            req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0"})
            with urllib.request.urlopen(req, timeout=6) as resp:
                data = json.loads(resp.read().decode("utf-8", errors="replace"))
                tag = str(data.get("tag_name") or "").strip()
            if not tag:
                raise ValueError("未找到版本号")
            # 源数据可能滞后（如 GitHub 只含旧 Release）：低于当前版本的结果不可信，
            # 跳过该源继续尝试下一源，避免误报“已是最新”。
            if _version_greater(APP_VERSION, tag):
                raise ValueError("版本数据滞后 (%s) " % tag)
            page = str(data.get("html_url") or "").strip() or fallback_pages.get(name, url)
            return {"ok": True, "latest": tag, "url": page, "source": name}
        except Exception as e:
            last = e
    return {"error": "无法获取有效版本信息: %s" % last}
result = fetch_latest()
if result.get("ok"):
    result["current"] = APP_VERSION
    result["update_available"] = _version_greater(result["latest"], APP_VERSION)
return result

def start_update(self, tag: str = ""):
    """开始自动更新：下载新版安装包 → 静默安装 → 自动重启到新版本。
    tag 为 check_update 返回的 latest（如 "v0.21.13"），留空则自动先查一次。
    立即返回 {"started": True, "tag": ...}，下载进度与结果通过
    window.__lnoteUpdateCb(payload) 回调推送：
    {"state": "downloading", "percent": 0-100| -1 未知, "received", "total"}
    → {"state": "ready", "path"} → {"state": "installing"};
    任一步失败推 {"ok": False, "error": ...}。"""
    if not str(tag or "").strip():
        chk = self.check_update()
        tag = str((chk or {}).get("latest") or "").strip()
    tag = str(tag or "").strip()
    if not tag:
        return {"error": "未指定要更新的版本号"}
    if getattr(self, "_updating", False):
        return {"error": "已有更新任务进行中，请稍候"}
    def push(payload):
        js = "if (window.__lnoteUpdateCb) window.__lnoteUpdateCb(%s);" % json.dumps(
            payload, ensure_ascii=False
        )
        try:
            if self._window:
                self._window.evaluate_js(js)
        except Exception:
            pass
    self._updating = True
    def worker():
        import urllib.request

```

```

import tempfile
import subprocess
try:
    url = _resolve_update_asset(tag)
    push({"ok": True, "state": "downloading", "percent": 0, "tag": tag})
    target = os.path.join(
        tempfile.gettempdir(), "L.Note-setup-%s.exe" % tag
    )
    partial = target + ".part"
    for p in (partial, target):
        if os.path.exists(p):
            try:
                os.remove(p)
            except Exception:
                pass
    req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0"})
    with urllib.request.urlopen(req, timeout=20) as resp:
        length = resp.headers.get("Content-Length")
        total = int(length) if length and str(length).isdigit() else 0
        received = 0
        with open(partial, "wb") as f:
            while True:
                chunk = resp.read(64 * 1024)
                if not chunk:
                    break
                f.write(chunk)
                received += len(chunk)
                percent = int(received * 100 / total) if total > 0 else -1
                push({
                    "ok": True,
                    "state": "downloading",
                    "percent": percent,
                    "received": received,
                    "total": total,
                })
    os.replace(partial, target)
    push({"ok": True, "state": "ready", "path": target})
    time.sleep(0.8)
# —— 静默安装 + 装完自动重启（由独立守护进程接力完成）——
# 旧实现只 Popen 安装器后 sleep 1.2s 便销毁窗口，问题有二：
# 1) 安装器按 Inno 默认目录装到 {localappdata}\Programs\L.Note,
# 而用户实际运行的是 D:\Program Files (x86)\L.Note 等自选
# 目录 → 装完找不到新版本；2) 无任何组件等安装结束再拉起
# 新版本，重启仅依赖安装器 [Run] 且目标目录还不匹配。
# 新实现：
# 1) 显式 /DIR 指向「当前正在运行的 exe」目录 → 原地升级；
# 2) 以 %TEMP% 中的 exe 副本（守护进程）等安装结束并自动启动
# 新版本 → 不再需要安装器 [Run] 或旧进程自行重启；
# 3) 新版本启动时前端 consume_update_result 读取结果并 toast。
# 若当前运行目录不可写/为空则回退 Inno 默认目录（开发环境）。

```

```

        if getattr(sys, "frozen", False):
            install_dir = os.path.dirname(os.path.abspath(sys.executable))
        else:
            install_dir = os.path.join(
                os.environ.get("LOCALAPPDATA") or os.path.expanduser("~"),
                "Programs", "L. Note",
            )
        try:
            os.makedirs(install_dir, exist_ok=True)
        except Exception:
            pass
        _write_update_pending(tag)
        guard_cmd = _spawn_update_guard([
            "--update-guard",
            "--target", target,
            "--dir", install_dir,
            "--exe", "L. Note.exe",
            "--tag", tag,
        ])
        _debug_log("[update] launch update guard: " + repr(guard_cmd))
        _launch_detached(guard_cmd)
        push({"ok": True, "state": "installing"})
        # 稍候片刻让 guard 真正起来, 再销毁窗口退出主进程;
        # 之后窗口由 guard 在安装完成后自动重启到新版本。
        time.sleep(1.2)
        try:
            if self._window:
                self._window.destroy()
        except Exception:
            pass
    except Exception as e:
        push({"ok": False, "error": "更新失败: %s" % e})
    finally:
        self._updating = False
        threading.Thread(target=worker, daemon=True).start()
        return {"started": True, "tag": tag}

# ----- 编码工具 -----
_UI_ENCS = {
    "utf-8": "UTF-8",
    "utf-8-sig": "UTF-8 (BOM)",
    "utf-16-le": "UTF-16LE",
    "utf-16-be": "UTF-16BE",
    "gb18030": "GB18030",
    "big5": "BIG5",
}

@staticmethod
def _ui_enc(codec: str) -> str:
    return LNoteApi._UI_ENCS.get(codec, codec)

@staticmethod
def _codec_from_ui(ui: str) -> str:

```

```
for codec, label in LNoteApi._UI_ENCS.items():
    if label == ui or codec == ui:
        return codec
return "utf-8"

def _encode_text(self, content: str, encoding: str) -> bytes:
    codec = self._codec_from_ui(encoding)
    return content.encode(codec, errors="replace")

@staticmethod
def _detect_encoding(raw: bytes) -> str:
    if raw.startswith(b"\xef\xbb\xbf"):
        return "utf-8-sig"
    if raw.startswith(b"\xff\xfe\x00\x00"):
        return "utf-32-le"
    if raw.startswith(b"\x00\x00\xfe\xff"):
        return "utf-32-be"
    if raw.startswith(b"\xff\xfe"):
        return "utf-16-le"
    if raw.startswith(b"\xfe\xff"):
        return "utf-16-be"
    try:
        raw.decode("utf-8")
        return "utf-8"
    except UnicodeDecodeError:
        pass
    try:
        raw.decode("gb18030")
        return "gb18030"
    except UnicodeDecodeError:
        pass
    return "latin-1"

def _ask_save_path(self, filename: str, file_types=None):
    if not self._window:
        return None

    # file_types: tuple of (描述, pattern) 序列; 为空时只用单过滤器「所有文件」
    if not file_types:
        file_types = ("所有文件 (*.*)",)
    # pywebview 接受 tuple/list/string; 统一转 tuple, 避免 None 或空 list 报错
    if isinstance(file_types, str):
        file_types = (file_types,)
    if isinstance(file_types, list):
        file_types = tuple(file_types)
    result = self._window.create_file_dialog(
        webview.SAVE_DIALOG,
        save_filename=filename,
        file_types=file_types,
    )
    # SAVE_DIALOG 返回 str 或 tuple, 取消时为 None / 空
    if isinstance(result, (list, tuple)):
        return result[0] if result else None
    if isinstance(result, str):
```

```

        return result
    return None

_MIME_MAP = {
    ".png": "image/png", ".jpg": "image/jpeg", ".jpeg": "image/jpeg",
    ".gif": "image/gif", ".webp": "image/webp", ".bmp": "image/bmp",
    ".svg": "image/svg+xml", ".ico": "image/x-icon",
}

def _guess_mime(path: str) -> str:
    ext = os.path.splitext(path)[1].lower()
    return _MIME_MAP.get(ext, "application/octet-stream")

def _looks_chinese(text: str) -> bool:
    """粗判文本是否以中文为主: 含 CJK 汉字且数量不小于英文字母数。"""
    cjk = sum(1 for ch in text if "\u4e00" <= ch <= "\u9fff")
    latin = sum(1 for ch in text if ch.isascii() and ch.isalpha())
    return cjk > 0 and cjk >= max(latin, 1)

# ---- 自动更新: 安装标记文件与守护进程 ----
# 更新分两段接力:
# 1) 主程序 start_update 下载安装包后, 写入 update-pending.json 标记,
#    再以独立守护进程 (见 _run_update_guard) 启动安装器并销毁自身窗口;
# 2) 守护进程静默运行安装器 (/DIR 原地升级到当前 exe 所在目录), 结束后
#    把结果写入 update-result.json, 成功则拉起新版本; 新版本启动时由前端
#    调用 consume_update_result 读取并删除该标记, toast 「已更新到 vX」。
def _update_pending_path() -> str:
    return os.path.join(_lnote_data_dir(), "update-pending.json")

def _update_result_path() -> str:
    return os.path.join(_lnote_data_dir(), "update-result.json")

def _write_update_pending(tag: str):
    """记录「正在升级到 tag」标记 (安装在失败/中断时留痕)。"""
    try:
        os.makedirs(_lnote_data_dir(), exist_ok=True)
        with open(_update_pending_path(), "w", encoding="utf-8") as f:
            json.dump({"tag": str(tag or ""), "ts": time.time()}, f, ensure_ascii=False)
    except Exception as e:
        _debug_log("[update] write pending failed: %s" % e)

def _write_update_result(ok: bool, version: str, error: str = ""):
    """写入更新结果标记, 供新版本启动时消费并 toast。"""
    try:
        os.makedirs(_lnote_data_dir(), exist_ok=True)
        with open(_update_result_path(), "w", encoding="utf-8") as f:
            json.dump({
                "ok": bool(ok),
                "version": str(version or ""),
                "error": str(error or ""),
                "ts": time.time(),
            }, f, ensure_ascii=False)
    except Exception as e:
        _debug_log("[update] write result failed: %s" % e)

def _read_update_result():
    """读取并删除更新结果标记; 无标记返回 None。"""
    path = _update_result_path()

```

```

try:
    if not os.path.isfile(path):
        return None
    with open(path, "r", encoding="utf-8") as f:
        data = json.load(f)
    os.remove(path)
    return {
        "ok": bool(data.get("ok")),
        "version": str(data.get("version") or ""),
        "error": str(data.get("error") or ""),
    }
except Exception as e:
    _debug_log("[update] read result failed: %s" % e)
    try:
        os.remove(path)
    except Exception:
        pass
    return None

def _is_ipc_alive() -> bool:
    """探测主编辑器实例是否仍在运行（IPC 端口可连即认为存活）。"""
    s = None
    try:
        s = socket.create_connection((IPC_HOST, IPC_PORT), timeout=0.4)
        return True
    except Exception:
        return False
    finally:
        if s is not None:
            try:
                s.close()
            except Exception:
                pass

def _launch_detached(cmd, cwd=None):
    """以独立进程组启动 cmd（不等待、不继承控制台）。"""
    try:
        flags = 0
        if sys.platform.startswith("win"):
            flags = 0x00000008 | 0x00000200 # DETACHED_PROCESS | CREATE_NEW_PROCESS_GROUP
        subprocess.Popen(
            cmd,
            cwd=cwd,
            close_fds=True,
            creationflags=flags,
        )
    except Exception as e:
        _debug_log("[update] detached launch failed: %s" % e)

def _spawn_update_guard(args):
    """构造「更新守护进程」启动命令。

    打包（frozen）模式：把当前 exe 复制到 %TEMP% 再用副本运行。副本不在
    安装目录内，Inno CloseApplications=force 关闭不到它，才能等安装结束

```

后自动拉起新版本；开发模式直接 `python main.py`（解释器不占用安装目录）。

"""

```
script = os.path.abspath(__file__)
if not getattr(sys, "frozen", False):
    return [sys.executable, script] + args
exe = sys.executable
tmp = os.path.join(tempfile.gettempdir(), "L.Note-update-guard.exe")
try:
    if os.path.exists(tmp):
        try:
            os.remove(tmp) # 上一轮残留副本（若已退出则可删）
        except Exception:
            pass
    shutil.copy2(exe, tmp)
    return [tmp] + args
except Exception as e:
    _debug_log("[update] guard copy failed, fallback to self exe: %s" % e)
    return [exe] + args
```

def _run_update_guard(argv):

"""更新守护进程（--update-guard）入口：等安装完成 → 自动拉起新版本。

由主程序在下载完成后以独立进程启动（打包模式下是 %TEMP% 中的 exe 副本）：

- 1) 等待旧主实例退出（窗口销毁后进程结束；超时不阻塞，安装器 CloseApplications=force 会兜底关闭仍在运行的旧进程）；
- 2) 静默运行安装器（/SILENT 会保留 Inno 原生安装进度窗口，用户可见），显式 /DIR 指向当前 exe 目录做「原地升级」并 /LOG 留痕；
- 3) 安装成功 → 写结果标记并启动新版本；失败 → 写失败标记并尽力恢复旧 exe。

"""

```
def arg(name, default=None):
    for i, a in enumerate(argv):
        if a == name and i + 1 < len(argv):
            return argv[i + 1]
    return default
target = arg("--target")
install_dir = arg("--dir")
exe_name = arg("--exe") or "L.Note.exe"
tag = arg("--tag") or ""
if not target or not install_dir or not os.path.isfile(target):
    _debug_log("[guard] missing or invalid args: " + repr(argv))
    return 2
_debug_log("[guard] start target=%s dir=%s exe=%s tag=%s"
            % (target, install_dir, exe_name, tag))
# 1) 等旧主实例退出（最多 ~8s），尽量让安装器替换文件时无进程占用
try:
    for _ in range(40):
        if not _is_ipc_alive():
            break
        time.sleep(0.2)
except Exception:
    pass
# 2) 静默安装：/DIR 原地升级 + /LOG 留痕。
```

```

        while (kk < n && isBlank(src[kk])) kk++;
        if (kk < n && isWordStart(src[kk])) continue;
        break;
    }
    for (const nm of names) bindings.set(nm, { kind: 'var', depth: 0 });
}
}
prevSignificant = word; i = j; continue;
}
if (!isBlank(c)) prevSignificant = c;
i++;
}
return { bindings: [...bindings.keys()], refs: [...refs] };
}
module.exports = { KEYWORDS, scan };
// ===== tools/publish-release.js =====
/*
 * tools/publish-release.js —— L.Note 双平台 (GitHub + Gitee) Release 统一发布 / 一致性校验脚本
 *
 * 背景: 曾发生 GitHub Release body 只写了 268 字摘要、而 Gitee 是完整 RELEASE-NOTES.md;
 *      GitHub tag 指向仓库初始 commit、与 Gitee 的实际发布 commit 不一致。
 *      本脚本把「正文 = RELEASE-NOTES.md 全量」「附件 = 两个 exe」「标题 = tag 提交信息」
 *      固化为唯一口径, 双平台走同一条代码路径, 杜绝手工差异。
 *
 * 正文约定: 以仓库根目录 RELEASE-NOTES.md 的整份内容作为 Release body (历史各版即如此,
 *           v0.21.15/16 双平台 body 均为该文件全量); 发送前统一把 \r\n 归一化为 \n。
 *
 * 用法:
 *   node tools/publish-release.js <tag>           # 校验模式: 本地 + 双平台只读检查, 不写任何远程
 *   node tools/publish-release.js <tag> --apply    # 落地: 缺失则创建、已存在则更新 body/标题/补齐附件, 随后自动复检
 *   node tools/publish-release.js <tag> --github-only # 只处理 GitHub
 *   node tools/publish-release.js <tag> --gitee-only  # 只处理 Gitee
 *
 * 认证:
 *   - GitHub: 使用 gh CLI (gh auth status 需已登录)
 *   - Gitee: GITEE_TOKEN 环境变量, 或仓库根目录 .gitee_token 文件 (该文件已 gitignore, 禁止入库)
 *
 * 前置门禁 (--apply 时强制):
 *   1. tag 在本地存在; 若平台尚无该 release, 则该平台远端 tag 必须与本地 tag 对象一致 (缺失即中止)
 *   2. RELEASE-NOTES.md 与 release/RELEASE-NOTES.md 完全一致 (publish.js 会同步)
 *   3. RELEASE-NOTES.md 首标题必须是 # L.Note <tag> 发布说明
 *   4. 两个发布 exe 存在且 SHA256 与 SHA256SUMS.txt 一致
 *
 * 退出码: 校验全部通过为 0, 任一 FAIL 为 1 (--apply 结束时若复检失败同样为 1)。
 */
'use strict';
const fs = require('fs');
const os = require('os');
const path = require('path');
const crypto = require('crypto');

```

```

const { spawnSync } = require('child_process');
const ROOT = path.resolve(__dirname, '..');
const APP = 'L. Note';
const UA = 'Mozilla/5.0 L.Note-publish-release';
const MAX_RETRY = 3;
// -----
// 参数
// -----

const args = process.argv.slice(2);
const TAG = (args.find(a => /^v\d+\.\d+\.\d+$/ .test(a)) || '').trim();
const APPLY = args.includes('--apply');
let PLATFORMS = ['github', 'gitee'];
if (args.includes('--github-only')) PLATFORMS = ['github'];
if (args.includes('--gitee-only')) PLATFORMS = ['gitee'];
const fails = [];
const warns = [];
function fail(msg) { console.error('[release] 错误: ' + msg); process.exit(1); }
function warn(msg) { warns.push(msg); console.warn('    • WARN ' + msg); }
function bad(scope, msg) { fails.push(scope + ' — ' + msg); console.error('    X FAIL ' + scope + ': ' + msg); }
function ok(scope, msg) { console.log('    ✓ PASS ' + scope + (msg ? ': ' + msg : '')); }
function info(scope, msg) { console.log('    • INFO ' + scope + (msg ? ': ' + msg : '')); }
if (!TAG) fail('用法: node tools/publish-release.js <vX.Y.Z> [--apply] [--github-only|--gitee-only]');
const VERSION = TAG.replace(/^v/, '');
const SETUP_EXE = APP + '-setup-' + TAG + '.exe';
const WIN64_EXE = APP + '-' + TAG + '-win64.exe';
const ASSETS = [SETUP_EXE, WIN64_EXE];
// -----
// 基础工具
// -----

function git(args) {
    return spawnSync('git', args, { cwd: ROOT, encoding: 'utf8', maxBuffer: 64 * 1024 * 1024 });
}

function sleep(ms) { return new Promise(res => setTimeout(res, ms)); }
async function withRetry(label, fn) {
    let lastErr = null;
    for (let i = 1; i <= MAX_RETRY; i++) {
        try { return await fn(); }
        catch (e) {
            lastErr = e;
            if (i < MAX_RETRY) { console.log('    • RETRY ' + label + ' (第 ' + i + ' 次失败: ' + e.message + ')'); await sleep(2000 * i); }
        }
    }
    throw lastErr;
}

function normBody(text) {
    return String(text)
        .replace(/\r\n/g, '\n').replace(/\r/g, '\n')
        .replace(/\t/g, ' ')
        .replace(/\n{3,}/g, '\n\n')

```

```

    .trim();
}

function sha256of(file) {
    return crypto.createHash('sha256').update(fs.readFileSync(file)).digest('hex').toUpperCase();
}

function parseSums() {
    const map = {};
    const p = path.join(ROOT, 'SHA256SUMS.txt');
    if (!fs.existsSync(p)) return map;
    const re = /SHA256\((.*)\)\s*=\s*([0-9A-F]{64})/g;
    let m;
    const text = fs.readFileSync(p, 'utf8');
    while ((m = re.exec(text))) map[m[1]] = m[2];
    return map;
}

// -----
// 平台识别（不硬编码 owner/repo，从远端 URL 解析）
// -----

function configGet(name) {
    const r = git(['config', '--get', name]);
    return r.status === 0 ? r.stdout.trim() : null;
}

const GITHUB_URL = configGet('remote.github.url') || configGet('remote.origin.url');
const GITEE_URL = configGet('remote.origin.url');
if (!GITHUB_URL || !/github\.com/.test(GITHUB_URL)) fail('无法定位 GitHub 远端 (remote.github.url 缺失)');
if (!GITEE_URL || !/gitee\.com/.test(GITEE_URL)) fail('无法定位 Gitee 远端 (remote.origin.url 缺失)');
const GH_REPO = (GITHUB_URL.match(/github\.com[:\/](^\/|\/\^[^\/]+?)\.git)?$/ || [])[1];
if (!GH_REPO) fail('GitHub 远端 URL 无法解析');
const giteeSeg = GITEE_URL.replace(/\.git$/i, '').replace(/\/+$/, '').split('/');
const GITEE = { owner: giteeSeg[giteeSeg.length - 2], repo: giteeSeg[giteeSeg.length - 1] };
console.log('release' + TAG + '  GitHub=' + GH_REPO + '  Gitee=' + GITEE.owner + '/' + GITEE.repo
    + (APPLY ? ' (--apply 落地)' : ' (校验模式, 只读)');

// -----
// Gitee token 解析 (env → 本地文件, 禁止入库)
// -----

let giteeToken = process.env.GITEE_TOKEN || null;
if (!giteeToken) {
    const p = path.join(ROOT, '.gitee_token');
    if (fs.existsSync(p)) giteeToken = fs.readFileSync(p, 'utf8').trim();
}

if (!giteeToken && PLATFORMS.includes('gitee')) {
    warn('未找到 Gitee token: 请设置环境变量 GITEE_TOKEN, 或在仓库根目录创建 .gitee_token 文件 (已 gitignore)');
}

// -----
// GitHub 访问 (gh CLI)
// -----

async function ghJson(jsonArgs) {
    return withRetry('gh' + jsonArgs[0] + ' ' + (jsonArgs[1] || ''), () => {
        const r = spawnSync('gh', jsonArgs, { encoding: 'utf8', maxBuffer: 64 * 1024 * 1024 });
        if (r.status !== 0) throw new Error((r.stderr || r.stdout || '').trim().split('\n').pop() || 'gh 退出码' + r.status);
    });
}

```

```

    return JSON.parse(r.stdout);
  });
}
async function ghRun(args) {
  return withRetry('gh' + args[0] + ' ' + (args[1] || ''), () => {
    const r = spawnSync('gh', args, { encoding: 'utf8', maxBuffer: 64 * 1024 * 1024 });
    if (r.status !== 0) throw new Error((r.stderr || r.stdout || '').trim().split('\n').pop() || 'gh 退出码' + r.status);
    return true;
  });
}
async function ghReleaseInfo(tag) {
  return ghJson(['release', 'view', tag, '--repo', GH_REPO, '--json', 'tagName,name,isDraft,body,assets']);
}
async function ghCreateOrUpdate(release, title, body) {
  const tmp = path.join(os.tmpdir(), 'lnote-rn-' + TAG + '.md');
  fs.writeFileSync(tmp, body + '\n', 'utf8');
  try {
    if (!release) {
      try {
        await ghRun(['release', 'create', TAG, '--repo', GH_REPO, '--title', title, '--notes-file', tmp]);
        ok('github', 'release 已创建 (标题/正文为统一口径)');
      } catch (e) {
        if (!/already exists/i.test(e.message)) throw e;
        await ghRun(['release', 'edit', TAG, '--repo', GH_REPO, '--title', title, '--notes-file', tmp]);
        ok('github', 'release 已存在 (并发/重试残留), 标题/正文已更新为统一口径');
      }
    } else {
      await ghRun(['release', 'edit', TAG, '--repo', GH_REPO, '--title', title, '--notes-file', tmp]);
      ok('github', 'release 标题/正文已更新为统一口径');
    }
  } finally {
    try { fs.unlinkSync(tmp); } catch (e) { /* 忽略 */ }
  }
  return ghReleaseInfo(TAG);
}
async function ghRemoteTagRef(tag) {
  const j = await ghJson(['api', 'repos/' + GH_REPO + '/git/refs/tags/' + tag]);
  return { sha: j.object.sha, type: j.object.type };
}
// -----
// Gitee 访问 (REST API v5)
// -----
const GITEE_API = 'https://gitee.com/api/v5';
async function giteeFetch(pathname, init) {
  if (!giteeToken) throw new Error('缺少 Gitee token');
  const sep = pathname.includes('?') ? '&' : '?';
  const url = GITEE_API + pathname + sep + 'access_token=' + encodeURIComponent(giteeToken);
  const base = Object.assign({ headers: { 'User-Agent': UA, 'Accept': 'application/json' } }, init || {});
  return withRetry('gitee' + init.method + ' ' + pathname, async () => {
    const res = await fetch(url, base);

```

```

    const text = await res.text();
    let json = null;
    try { json = JSON.parse(text); } catch (e) { /* 非 JSON */ }
    if (!res.ok) {
        const msg = json ? (json.message || json.error || JSON.stringify(json)) : ('HTTP ' + res.status);
        throw new Error(msg);
    }
    return json;
});
}

async function giteeGetRelease(tag) {
    try {
        return await giteeFetch('/repos/' + GITEE.owner + '/' + GITEE.repo + '/releases/tags/' + encodeURIComponent(tag), { method:
'GET' });
    } catch (e) {
        if (/404|not found|不存在/i.test(e.message)) return null;
        throw e;
    }
}

async function giteeCreateOrUpdate(release, title, body) {
    const commit = git(['rev-list', '-n', '1', TAG]);
    const targetCommitish = commit.status === 0 && commit.stdout.trim() ? commit.stdout.trim() : '';
    const payload = JSON.stringify({ tag_name: TAG, target_commitish: targetCommitish, name: title, body: body });
    if (!release) {
        const j = await giteeFetch('/repos/' + GITEE.owner + '/' + GITEE.repo + '/releases', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json; charset=utf-8' },
            body: payload,
        });
        ok('gitee', 'release 已创建 (标题/正文为统一口径)');
        return j;
    }
    const j = await giteeFetch('/repos/' + GITEE.owner + '/' + GITEE.repo + '/releases/' + release.id, {
        method: 'PATCH',
        headers: { 'Content-Type': 'application/json; charset=utf-8' },
        body: payload,
    });
    ok('gitee', 'release 标题/正文已更新为统一口径');
    return j;
}

async function giteeRemoteTag(tag) {
    return withRetry('gitee ls-remote ' + tag, () => {
        const r = git(['ls-remote', '--tags', 'origin', 'refs/tags/' + tag]);
        if (r.status !== 0) throw new Error((r.stderr || '').trim() || 'ls-remote 失败');
        const line = (r.stdout || '').trim();
        return line ? line.split('\t')[0] : null;
    });
}

// -----
// 本地前置校验

```

```
// -----
const localAssets = {};
for (const name of ASSETS) {
  localAssets[name] = path.join(ROOT, 'release', name);
  if (!fs.existsSync(localAssets[name])) bad('local', '缺少发布产物 release/' + name);
}

function localGates() {
  console.log('\n== 本地门禁 ==');
  const tr = git(['rev-parse', '--verify', TAG]);
  if (tr.status !== 0) fail('本地不存在 tag ' + TAG + ', 请先 git tag ' + TAG);
  ok('local', 'tag ' + TAG + ' 存在: ' + tr.stdout.trim());
  const sums = parseSums();
  for (const name of ASSETS) {
    const p = localAssets[name];
    if (!fs.existsSync(p)) continue;
    const want = sums[name];
    if (!want) { warn('SHA256SUMS.txt 缺少 ' + name + ' 的校验和'); continue; }
    if (sha256of(p) !== want) bad('local', name + ' SHA256 与 SHA256SUMS.txt 不一致');
    else ok('local', name + ' SHA256 与 SHA256SUMS.txt 一致');
  }
  const rnP = path.join(ROOT, 'RELEASE-NOTES.md');
  const rnCopyP = path.join(ROOT, 'release', 'RELEASE-NOTES.md');
  if (!fs.existsSync(rnP)) fail('缺少 RELEASE-NOTES.md');
  if (fs.existsSync(rnCopyP) && normBody(fs.readFileSync(rnP, 'utf8')) !== normBody(fs.readFileSync(rnCopyP, 'utf8'))) {
    bad('local', 'release/RELEASE-NOTES.md 与根目录不一致 (请先跑 node tools/publish.js ' + VERSION + ' --apply 同步)');
  }
  const rnText = fs.readFileSync(rnP, 'utf8');
  if (!rnText.startsWith('# L.Note ' + TAG + ' 发布说明')) {
    bad('local', 'RELEASE-NOTES.md 首标题不是 "# L.Note ' + TAG + ' 发布说明"');
  }
  const subject = git(['log', '-1', '--format=%s', TAG]);
  const title = subject.status === 0 && subject.stdout.trim() ? subject.stdout.trim() : TAG;
  ok('local', '标题口径 = tag 提交信息: ' + title);
  return { rnText: rnText, title: title };
}

// -----
// 附件校验 (存在 + 大小一致; 缺失时只读模式 FAIL、落地模式补传)
// -----
async function checkGithubAssets(info) {
  const assets = (info.assets || []).map(a => a.name);
  for (const name of Object.keys(localAssets)) {
    const hit = (info.assets || []).find(a => a.name === name);
    if (!hit) {
      if (APPLY) {
        await withRetry('gh upload ' + name, () => {
          const r = spawnSync('gh', ['release', 'upload', TAG, '--repo', GH_REPO, localAssets[name], '--clobber'], { encoding: 'utf8' });
          if (r.status !== 0) throw new Error((r.stderr || r.stdout || '').trim() || '上传失败');
        });
        ok('github', '附件已上传: ' + name);
      }
    }
  }
}
```

```

    } else {
      bad('github', '缺少附件' + name + ' (--apply 可补传)');
    }
    continue;
  }
  const localSize = fs.statSync(localAssets[name]).size;
  if (hit.size !== null && Number(hit.size) !== localSize) bad('github', '附件' + name + ' 大小异常 (远端' + hit.size + ' ≠ 本地' + localSize + ')');
  else ok('github', '附件在远端且大小一致: ' + name);
}
const ghost = assets.filter(a => !Object.keys(localAssets).includes(a));
if (ghost.length) warn('github 存在额外附件 (不在两件套内, 不处理): ' + ghost.join(', '));
}

async function checkGiteeAssets(release) {
  const assets = release.assets || [];
  const names = assets.map(a => a.name);
  for (const name of Object.keys(localAssets)) {
    if (names.includes(name)) {
      ok('gitee', '附件在远端: ' + name);
      continue;
    }
    if (!APPLY) { bad('gitee', '缺少附件' + name + ' (--apply 可补传)'); continue; }
    const fd = new FormData();
    fd.append('file', new Blob([fs.readFileSync(localAssets[name])]), name);
    await giteeFetch('/repos/' + GITEE.owner + '/' + GITEE.repo + '/releases/' + release.id + '/attach_files', {
      method: 'POST',
      body: fd,
    });
    ok('gitee', '附件已上传: ' + name);
  }
  const ghost = names.filter(n => !Object.keys(localAssets).includes(n));
  if (ghost.length) warn('gitee 存在额外附件 (不在两件套内, 不处理): ' + ghost.join(', '));
}

// -----
// 平台校验 / 落地
// -----

async function runGithub(localBodyNorm, title) {
  console.log('\n== GitHub (' + GH_REPO + ') ==');
  const localSha = git(['rev-parse', TAG]).stdout.trim();
  let tagAligned = true;
  try {
    const ref = await ghRemoteTagRef(TAG);
    if (ref.type !== 'tag') info('github', '远端 tag 为 lightweight (本地为带注释 tag 时属正常差异)');
    if (ref.sha !== localSha) { bad('github', '远端 tag=' + ref.sha + ' ≠ 本地 tag=' + localSha + ' (tag-first: 远端 tag 必须与本地一致, 禁止在错误 commit 上发布)'); tagAligned = false; }
    else ok('github', '远端 tag 与本地一致 ' + localSha);
  } catch (e) {
    bad('github', '远端 tag 查询失败: ' + e.message);
    tagAligned = false;
  }
}

```

```

let rel = null;
try { rel = await ghReleaseInfo(TAG); }
catch (e) { if (!/not found/i.test(e.message)) bad('github', '查询 release 失败: ' + e.message); }
if (!rel) {
  info('github', 'release 不存在' + (APPLY ? ' (将创建)' : ' (--apply 可创建)'));
  if (!APPLY) { bad('github', 'release 不存在'); return; }
} else if (rel.isDraft) {
  bad('github', 'release 处于 draft 状态, --apply 无法解除, 请人工处理');
  return;
}
if (APPLY && !tagAligned) {
  bad('github', 'tag 与本地不一致, 中止写入: 请先修正远端 tag (推送正确 commit 或删除重建 release) 后再 --apply');
  return;
}
if (APPLY) rel = await ghCreateOrUpdate(rel, title, localBodyNorm);
if (normBody(rel.name || '') !== normBody(title)) bad('github', '标题不一致 (现: ' + rel.name + ')');
else ok('github', '标题 = ' + rel.name);
const relBodyNorm = normBody(rel.body || '');
if (relBodyNorm !== localBodyNorm) {
  bad('github', 'body 与 RELEASE-NOTES.md 不一致 (现 ' + (rel.body || '').length + ' 字, 应为 ' + localBodyNorm.length + ' 字)');
} else {
  ok('github', 'body 与 RELEASE-NOTES.md 全量一致 (' + localBodyNorm.length + ' 字)');
}
await checkGithubAssets(rel);
}

async function runGitee(localBodyNorm, title) {
  console.log('\n== Gitee (' + GITEE.owner + '/' + GITEE.repo + ') ==');
  const localSha = git(['rev-parse', TAG]).stdout.trim();
  const remoteSha = await giteeRemoteTag(TAG);
  let tagAligned = !!remoteSha;
  if (!remoteSha) bad('gitee', '远端不存在 tag ' + TAG + ' (tag-first: 请先 git push origin ' + TAG + ')');
  else if (remoteSha !== localSha) { bad('gitee', '远端 tag=' + remoteSha + ' ≠ 本地 tag=' + localSha + ' (tag-first: 远端 tag 必须与本地一致, 禁止在错误 commit 上发布)'); tagAligned = false; }
  else ok('gitee', '远端 tag 与本地一致 ' + localSha);
  if (!giteeToken) { bad('gitee', '缺少 token, 无法继续'); return; }
  const rel0 = await giteeGetRelease(TAG);
  if (!rel0) {
    info('gitee', 'release 不存在' + (APPLY ? ' (将创建)' : ' (--apply 可创建)'));
    if (!APPLY) { bad('gitee', 'release 不存在'); return; }
  }
  if (APPLY && !tagAligned) {
    bad('gitee', 'tag 与本地不一致, 中止写入: 请先修正远端 tag 后再 --apply');
    return;
  }
  const rel = APPLY ? await giteeCreateOrUpdate(rel0 || null, title, localBodyNorm) : rel0;
  if (normBody(rel.name || '') !== normBody(title)) bad('gitee', '标题不一致 (现: ' + rel.name + ')');
  else ok('gitee', '标题 = ' + rel.name);
  const relBodyNorm = normBody(rel.body || '');
  if (relBodyNorm !== localBodyNorm) {

```

```

    bad('gitee', 'body 与 RELEASE-NOTES.md 不一致 (现 ' + (rel.body || '').length + ' 字, 应为 ' + localBodyNorm.length + '
字)');
  } else {
    ok('gitee', 'body 与 RELEASE-NOTES.md 全量一致 (' + localBodyNorm.length + ' 字)');
  }

  await checkGiteeAssets(rel);
}

// -----
// 主流程
// -----

(async function main() {
  const gates = localGates();
  const localBodyNorm = normBody(gates.rnText);
  const localSize = {};
  for (const name of Object.keys(localAssets)) {
    if (fs.existsSync(localAssets[name])) localSize[name] = fs.statSync(localAssets[name]).size;
  }
  if (PLATFORMS.includes('github')) await runGithub(localBodyNorm, gates.title);
  if (PLATFORMS.includes('gitee')) await runGitee(localBodyNorm, gates.title);
  console.log('\n== 汇总 ==');
  if (fails.length) {
    console.error('结果: FAIL (' + fails.length + ' 项)' + (APPLY ? ', 请修复上述问题后重试' : ', --apply
可修复标题/正文/附件类差异'));
    process.exitCode = 1;
    return;
  }
  console.log('结果: PASS —— 双平台 Release 与 RELEASE-NOTES.md 口径完全一致');
})();
}).catch(e => {
  console.error('[release] 未捕获异常: ' + (e && (e.stack || e.message) || e));
  process.exitCode = 1;
});

// ===== tools/publish.js =====
/*
 * tools/publish.js —— L.Note 发布前「版本印章 + 文档同步」固化脚本
 *
 * 职责 (执行边界: 不含代码构建 / Inno Setup 打包 / git 提交与推送, 均按 project_rules.md 人工执行):
 * 1. 校验并读取当前版本 (main.py 的 APP_VERSION 为准, 须与 package.json 一致)
 * 2. 版本印章: 对“单版本载体”做全文 vCUR/CUR → vNEW/NEW 替换:
 *    main.py / package.json / src-app/27-about.js / installer/LNote.iss / blink-guide.html
 *    (js/app.js 为构建产物, 不直接改, 需按构建链重新生成)
 * 3. 累计文档不做全文替换, 只插入新条目:
 *    - CHANGELOG.md: 在顶部 (首条 “## [v…]” 标题之前) 插入 ## [vNEW] - 日期 + ### 变更
 *    - RELEASE-NOTES.md: 只重写“## 更新日志 (本版亮点)”标题之前的部分 (标题/文件名/用法等
 *      单版本信息), 并在该节顶部插入 **vNEW** 亮点块, 历史正文一律不动
 *    - download.html: 只重写“更新日志”小节之前的部分 (徽标/下载按钮/快速开始/certutil),
 *      并在其更新日志区顶部插入新的 <div class="changelog"> 条目, 历史条目一律不动
 * 4. docs/06-功能列表.md: 仅替换顶部「版本 / 更新日期」元信息行
 * 5. 读取 release/ 下两个新版本 exe 的 SHA256, 重建 SHA256SUMS.txt (根目录 + release/ 副本),
 *    并以“旧哈希 → 新哈希”方式替换 download.html 中的两处哈希
 * 6. 同步副本: index.html = download.html 的整份拷贝; release/download.html / release/RELEASE-NOTES.md

```

```

*      与根目录处理后结果保持整份一致
*
* 用法:
*   node tools/publish.js <新版本号>          # 干跑: 只检查与预览, 不写任何文件
*   node tools/publish.js <新版本号> --apply    # 落地: 真正写文件
*   node tools/publish.js <新版本号> --apply --date 2026-09-06      # 指定日志日期 (缺省=今天)
*   node tools/publish.js <新版本号> --apply --notes tools/_rn-新.md  # 用笔记文件生成日志条目
*
* 注意: --notes 内容由发布人准备 (每行 "- 亮点..." 即可); 缺省时写入 TODO 占位, 正文留待人工补写。
* 落地前请确保 release/ 下两个新版本 exe 已就位 (哈希依赖它们计算)。
*/
'use strict';
const fs = require('fs');
const path = require('path');
const crypto = require('crypto');
const ROOT = path.resolve(__dirname, '..');
const APP = 'L.Note';
// -----
// 小工具
// -----
function fail(msg) {
    console.error('[publish] 错误: ' + msg);
    process.exit(1);
}
function read(p) {
    return fs.readFileSync(p, 'utf8');
}
function write(p, text) {
    fs.writeFileSync(p, text);
}
function sha256of(file) {
    const buf = fs.readFileSync(file);
    return crypto.createHash('sha256').update(buf).digest('hex').toUpperCase();
}
function pad2(n) {
    return String(n).padStart(2, '0');
}
function today() {
    const d = new Date();
    return d.getFullYear() + '-' + pad2(d.getMonth() + 1) + '-' + pad2(d.getDate());
}
function git(args) {
    const { spawnSync } = require('child_process');
    const r = spawnSync('git', args, { cwd: ROOT, encoding: 'utf8' });
    return r;
}
function eolOf(text) {
    return text.includes('\r\n') ? '\r\n' : '\n';
}
// -----

```

```
// 入口参数解析
// -----
const args = process.argv.slice(2);
const APPLY = args.includes('--apply');
let dateArg = null;
let notesArg = null;
const positionals = [];
for (let i = 0; i < args.length; i++) {
  const a = args[i];
  if (a === '--apply') continue;
  if (a === '--date') { dateArg = args[++i]; continue; }
  if (a === '--notes') { notesArg = args[++i]; continue; }
  if (a.startsWith('--')) fail('未知参数 ' + a);
  positionals.push(a);
}
if (positionals.length !== 1) {
  console.error('用法: node tools/publish.js <版本号> [--apply] [--date YYYY-MM-DD] [--notes 文件]');
  process.exit(1);
}
const NEW = positionals[0];
if (!/^\\d+\\.\\d+\\.\\d+$/.test(NEW)) fail('新版本号格式应为 X.Y.Z, 收到: ' + NEW);
const DATE = dateArg || today();
if (!/^\\d{4}-\\d{2}-\\d{2}$/.test(DATE)) fail('--date 格式应为 YYYY-MM-DD, 收到: ' + DATE);
let notesText = null;
if (notesArg) {
  const np = path.resolve(ROOT, notesArg);
  if (!fs.existsSync(np)) fail('--notes 文件不存在: ' + np);
  notesText = read(np).trim();
}
// -----
// 读取当前版本
// -----
const mainPy = read(path.join(ROOT, 'main.py'));
const m1 = mainPy.match(/APP_VERSION\\s*=\\s*["']([^\"]+)["']/);
if (!m1) fail('main.py 中未找到 APP_VERSION');
const CUR = m1[1];
const pkgJson = JSON.parse(read(path.join(ROOT, 'package.json')));
if (pkgJson.version !== CUR) {
  fail('版本不一致: package.json=' + pkgJson.version + ' ≠ main.py=' + CUR + ', 请先统一再发布');
}
if (!/^\\d+\\.\\d+\\.\\d+$/.test(CUR)) fail('当前版本号格式异常: ' + CUR);
if (CUR === NEW) fail('新版本与当前版本相同: ' + NEW);
// -----
// 计算涉及的路径集合
// -----
const P = {
  mainPy: 'main.py',
  pkgJson: 'package.json',
  aboutJs: path.join('src-app', '27-about.js'),
  liss: path.join('installer', 'LNote.iss'),
```

```
changelog: 'CHANGELOG.md',
feats: path.join('docs', '06-功能列表.md'),
rn: 'RELEASE-NOTES.md',
sums: 'SHA256SUMS.txt',
dl: 'download.html',
idx: 'index.html',
blink: 'blink-guide.html',
siteDl: path.join('release', 'download.html'),
siteRn: path.join('release', 'RELEASE-NOTES.md'),
siteSums: path.join('release', 'SHA256SUMS.txt'),
};
// 单版本载体文件：全文件出现的 vCUR / CUR 均属当前版信息，可整体替换为新版
const STAMP_FILES = [P.mainPy, P.pkgJson, P.aboutJs, P.liss, P.blink];
// release/ 下两个发布 exe（新版本，落地前必须已构建就位；仓库命名带 "v"）
const setupExe = APP + '-setup-v' + NEW + '.exe';
const win64Exe = APP + '-v' + NEW + '-win64.exe';
const exePaths = {
  setup: path.join(ROOT, 'release', setupExe),
  win64: path.join(ROOT, 'release', win64Exe),
};
// -----
// 汇总
// -----
const plan = []; // 已落地动作
const preview = []; // 干跑预览动作
const warnings = [];
function commit(abs, text, what) {
  if (APPLY) {
    write(abs, text);
    plan.push(what);
  } else {
    preview.push(what);
  }
}
// 全文 token 替换：vCUR → vNEW，再 CUR → NEW；返回替换后文本与总替换处数
function stampTokens(text) {
  let t = text;
  const c1 = t.split('v' + CUR).length - 1;
  t = t.split('v' + CUR).join('v' + NEW);
  const c2 = t.split(CUR).length - 1;
  t = t.split(CUR).join(NEW);
  return { text: t, total: c1 + c2 };
}
// -----
// 1) 单版本载体整体盖章
// -----
function stampFile(rel) {
  const abs = path.join(ROOT, rel);
  const raw = read(abs);
  if (rel === P.pkgJson) {
```

```

    const obj = JSON.parse(raw);
    if (obj.version !== CUR) {
        warnings.push('package.json 的 version 字段异常: ' + obj.version + ' (跳过盖章)');
        return null;
    }
    obj.version = NEW;
    const eol = eolOf(raw);
    const text = JSON.stringify(obj, null, 2).split('\n').join(eol) + eol;
    return { text: text, desc: 'package.json version 字段 ' + CUR + ' → ' + NEW };
}

const r = stampTokens(raw);
if (r.total === 0) {
    warnings.push('未发现当前版本号 ' + CUR + ' (跳过盖章): ' + rel);
    return null;
}

return { text: r.text, desc: '版本号 ' + CUR + ' → ' + NEW + ' (' + r.total + ' 处)' };
}

// -----
// 4) docs/06-功能列表.md: 仅替换顶部「版本 / 更新日期」元信息行
// -----

function stampFeatureHead() {
    const abs = path.join(ROOT, P.feats);
    const raw = read(abs);
    const lines = raw.split(/\r?\n/);
    const i = 2; // 第 3 行
    if (!lines[i] || !lines[i].includes(CUR)) {
        warnings.push('docs/06-功能列表.md 第 3 行未含当前版本 ' + CUR + ' (未改动)');
        return null;
    }
    const out = lines[i]
        .split('v' + CUR).join('v' + NEW)
        .split(CUR).join(NEW)
        .replace(/\d{4}-\d{2}-\d{2}/, DATE);
    lines[i] = out;
    return { text: lines.join(eolOf(raw)), desc: '功能列表第 3 行版本/日期更新为 v' + NEW + ' / ' + DATE };
}

// -----
// 3a) CHANGELOG.md: 在首条 "## \[v···" 标题之前插入新条目
// -----

function changelogBody() {
    return notesText || '> TODO: 发布前人工补充本版变更说明 (新增 / 修复 / 变更三分类) 与测试说明.';
}

function buildChangelogText() {
    const abs = path.join(ROOT, P.changelog);
    const raw = read(abs);
    const anchor = '## \[v'; // 匹配仓库风格 "## \[v0.21.15] - yyyy-mm-dd"
    const i = raw.indexOf(anchor);
    if (i === -1) {
        warnings.push('CHANGELOG.md 未找到 "## \[v···" 条目标题 (跳过插入)');
        return null;
    }

```

```
}
const entry = '## \\[v' + NEW + ']' - ' + DATE + '\\n\\n### 变更\\n\\n' + changelogBody();
const text = raw.slice(0, i) + entry + '\\n\\n' + raw.slice(i);
return { text: text, desc: 'CHANGELOG.md 顶部插入新条目 ## [v' + NEW + ']' - ' + DATE };
}
// -----
// 3b) RELEASE-NOTES.md: 只盖章「更新日志（本版亮点）」之前的部分 + 顶部插亮点块
// -----
function rnBulletBlock() {
    return notesText || ' - TODO: 发布前人工补充本版亮点条目';
}
function buildRnText() {
    const abs = path.join(ROOT, P.rn);
    const raw = read(abs);
    const hd = '## 更新日志（本版亮点）';
    const i = raw.indexOf(hd);
    if (i === -1) {
        warnings.push('RELEASE-NOTES.md 未找到「' + hd + '」标题（跳过更新）');
        return null;
    }
    const pre = raw.slice(0, i);
    const r = stampTokens(pre);
    if (r.total === 0) {
        warnings.push('RELEASE-NOTES.md 头部未发现当前版本号 ' + CUR + '（仅尝试插入亮点块，请人工核对）');
    }
    const j = raw.indexOf('**v' + CUR + '**', i);
    let text;
    let desc = 'RELEASE-NOTES.md 头部盖章（' + r.total + '处）+ 更新日志区顶部插入 **v' + NEW + '**';
    if (j === -1) {
        warnings.push('RELEASE-NOTES.md 未找到 "**v' + CUR + '**" 亮点块锚点（新块插在标题之后，请人工核对排版）');
        text = r.text + raw.slice(i, i + hd.length) + '\\n\\n**v' + NEW + '**\\n\\n' + rnBulletBlock() + raw.slice(i + hd.length);
    } else {
        const block = '**v' + NEW + '**\\n\\n' + rnBulletBlock();
        text = r.text + raw.slice(i, j) + block + '\\n\\n' + raw.slice(j);
    }
    return { text: text, desc: desc };
}
// -----
// 5) 哈希: release/ 下两个新 exe → SHA256SUMS.txt（根 + release/）
// -----
function hashes() {
    const out = {};
    const missing = [];
    for (const key of ['setup', 'win64']) {
        const f = exePaths[key];
        if (!fs.existsSync(f)) { missing.push(f); continue; }
        out[key] = sha256f(f);
    }
    return { out: out, missing: missing };
}
```

```
// 解析 “SHA256(<文件名>) = <64位大写HEX>” 列表为 { 文件名: 哈希 }
function parseSums(text) {
  const map = {};
  const re = /SHA256\((.*)\)\s*=\s*([0-9A-F]{64})/g;
  let m;
  while ((m = re.exec(text))) map[m[1]] = m[2];
  return map;
}

function readOldSums() {
  const p = path.join(ROOT, P.sums);
  if (!fs.existsSync(p)) {
    warnings.push('未找到旧 SHA256SUMS.txt, 无法定位 download.html 中的旧哈希 (跳过哈希替换)');
    return null;
  }
  return parseSums(read(p));
}

function syncSums(out) {
  const lines = [];
  if (out.setup) lines.push('SHA256(' + setupExe + ') = ' + out.setup);
  if (out.win64) lines.push('SHA256(' + win64Exe + ') = ' + out.win64);
  const text = lines.join('\r\n') + '\r\n';
  const desc = 'SHA256SUMS.txt 重建为 ' + NEW + ' 两条哈希';
  commit(path.join(ROOT, P.sums), text, desc + ' (根目录)');
  commit(path.join(ROOT, P.siteSums), text, desc + ' (release/)');
}

// 以旧哈希→新哈希方式替换 download.html 中的哈希
function replaceHashesInText(text, oldSums, out) {
  let t = text;
  let changed = 0;
  const oldFiles = {
    setup: APP + '-setup-v' + CUR + '.exe',
    win64: APP + '-v' + CUR + '-win64.exe',
  };
  for (const key of ['setup', 'win64']) {
    if (!out[key]) continue;
    const oldH = oldSums && oldSums[oldFiles[key]];
    if (!oldH) {
      warnings.push('旧 SHA256SUMS.txt 中未找到 ' + oldFiles[key] + ' 的哈希 (跳过该项替换)');
      continue;
    }
    const n = t.split(oldH).length - 1;
    if (n === 0) {
      warnings.push('download.html 中未找到旧哈希 ' + oldH + ' (' + oldFiles[key] + ', 跳过该项替换)');
      continue;
    }
    t = t.split(oldH).join(out[key]);
    changed += n;
  }
  return { text: t, changed: changed };
}
```

```
// -----
// 3c) download.html: 只盖章「更新日志」之前的部分 + 日志区顶部插新块 + 换哈希
// -----
function htmlSnippet() {
  if (!notesText) return 'TODO: 发布前人工补充本版亮点条目';
  return notesText
    .replace(/\s*[-*]\s+/gm, '')
    .replace(/\s*\n\s*/g, ', ')
    .trim();
}

function buildDlText(out, oldSums) {
  const abs = path.join(ROOT, P.dl);
  const raw = read(abs);
  const hd = '<h2>更新日志</h2>';
  const i = raw.indexOf(hd);
  if (i === -1) {
    warnings.push('download.html 未找到「更新日志」小标题（跳过更新）');
    return null;
  }
  const r = stampTokens(raw.slice(0, i));
  if (r.total === 0) {
    warnings.push('download.html 头部（更新日志之前）未发现当前版本号 ' + CUR + '（仅尝试插入日志块，请人工核对）');
  }
  let text = r.text + raw.slice(i);
  // 日志区顶部（首个 <div class="changelog"> 所在行行首）插入新块，旧块原有缩进保持不变
  const k = text.indexOf('<div class="changelog">', i);
  if (k === -1) {
    warnings.push('download.html 更新日志区未找到 <div class="changelog"> 结构（跳过插入）');
    return null;
  }
  const lineStart = text.lastIndexOf('\n', k) + 1;
  const block =
    '    <div class="changelog">\n' +
    '      <div class="v">v' + NEW + '</div>\n' +
    '      <ul>\n' +
    '        <li>' + htmlSnippet() + '</li>\n' +
    '      </ul>\n' +
    '    </div>\n';
  text = text.slice(0, lineStart) + block + text.slice(lineStart);
  // 哈希替换（仅头部区域存在旧哈希，全文替换即安全）
  if (out && (out.setup || out.win64)) {
    const h = replaceHashesInText(text, oldSums, out);
    text = h.text;
    if (h.changed === 0) warnings.push('download.html 哈希未发生替换，请人工核对');
  }
  return {
    text: text,
    desc: 'download.html 盖章（' + r.total + ' 处）' + ' + ' + '日志区插入 v' + NEW + ' 条目' + (out && (out.setup || out.win64) ?
'+哈希更新' : ''),
  };
};
```

```

}
// -----
// 6) 副本同步: index.html / release/ 下镜像整份一致
// -----
function copyFile(fromKey, toKey, what) {
  const src = path.join(ROOT, fromKey);
  const dst = path.join(ROOT, toKey);
  commit(dst, read(src), what);
}
// -----
// 干跑 / 落地
// -----
function main() {
  console.log(' [publish] 当前版本 ' + CUR + ' → 新版本 ' + NEW + (APPLY ? ' (落地 --apply) ' : ' (干跑, 不写文件)'));
  console.log(' [publish] 根目录: ' + ROOT);
  // 0) 工作区干净性提示 (不阻断)
  if (APPLY) {
    const st = git(['status', '--porcelain']);
    const dirty = (st.stdout || '').trim().split('\n').filter(Boolean);
    if (dirty.length) {
      console.log(' [publish] 注意: 工作区存在未提交改动 (' + dirty.length + ' 项), --apply 将叠加写入。');
    }
  }
  // 发布产物检查: 落地前必须两个新 exe 就位 (哈希依赖)
  const hres = hashes();
  const setupOk = fs.existsSync(exePaths.setup);
  const winOk = fs.existsSync(exePaths.win64);
  console.log(' [publish] 发布产物检查: ' + (setupOk ? '✓' : '✗') + ' ' + setupExe + ' '
    + (winOk ? '✓' : '✗') + ' ' + win64Exe);
  if (APPLY && !(setupOk && winOk)) {
    fail('release/ 下缺少新版本发布产物, 请先完成构建与打包 (哈希与下载页更新依赖它们)');
  }
  // 1) 单版本载体整体盖章
  for (const rel of STAMP_FILES) {
    const s = stampFile(rel);
    if (s) commit(path.join(ROOT, rel), s.text, s.desc + ' → ' + rel);
  }
  // 2) 功能列表头部
  const feat = stampFeatureHead();
  if (feat) commit(path.join(ROOT, P.feats), feat.text, feat.desc);
  // 3) 累计文档插条目 (CHANGELOG / RELEASE-NOTES / download.html)
  const cl = buildChangelogText();
  if (cl) commit(path.join(ROOT, P.changelog), cl.text, cl.desc);
  const rn = buildRnText();
  if (rn) commit(path.join(ROOT, P.rn), rn.text, rn.desc);
  const oldSums = readOldSums();
  const dl = buildDlText(hres.out, oldSums);
  if (dl) commit(path.join(ROOT, P.dl), dl.text, dl.desc);
  // 4) 哈希重建 (根 + release/)
  if (setupOk && winOk) {

```

```

    if (APPLY) {
      syncSums(hres.out);
    } else {
      preview.push('SHA256SUMS.txt 将重建为 setup=' + hres.out.setup.slice(0, 8) + '... / win64='
        + hres.out.win64.slice(0, 8) + '... (根目录 + release/)');
    }
  } else {
    warnings.push('发布产物缺失, 已跳过哈希重建与 download.html 哈希替换 (请先构建)');
  }
// 5) 副本同步 (整份拷贝)
copyFile(P.dl, P.idx, 'index.html 同步为 download.html 整份副本');
copyFile(P.rn, P.siteRn, 'release/RELEASE-NOTES.md 同步为根目录整份副本');
copyFile(P.dl, P.siteDl, 'release/download.html 同步为根目录整份副本 (修复历史遗留旧版)');
// 6) 汇总
for (const w of warnings) console.warn('[publish] 警告: ' + w);
if (!APPLY) {
  if (preview.length) {
    console.log('\n[publish] 干跑将执行 ' + preview.length + ' 项动作: ');
    for (const line of preview) console.log('  • ' + line);
  }
  console.log('\n[publish] 干跑结束: 未写任何文件。确认无误后执行:');
  console.log(' node tools/publish.js ' + NEW + ' --apply [--date ' + DATE + '][--notes 文件]');
  return;
}
console.log('\n[publish] 落地完成, 共 ' + plan.length + ' 项写入: ');
for (const line of plan) console.log('  • ' + line);
if (warnings.length) console.log('[publish] 存在 ' + warnings.length + ' 条警告, 请人工核对上述内容。');
console.log('\n[publish] 后续按 project_rules.md 人工执行: 构建链 → PyInstaller → Inno Setup → 推 GitHub/Gitee → 检出 gh-pages。');
}
main();
// ===== tools/refactor-rename.js =====
#!/usr/bin/env node
/* =====
* refactor-rename.js —— Phase 2 状态收拢的词法安全替换脚本
*
* 把 src-app/*.js 中散落的裸标识符 docs / activeId / previewOn /
* currentVisual 统一替换为 state.* (集中式状态对象):
*   docs          -> state.docs
*   activeId       -> state.activeId
*   previewOn      -> state.previewOn
*   currentVisual -> state.currentVisual
*
* 安全性 (词法扫描, 非正则全局替换):
*   1. 跳过字符串字面量 ('...' / 模板) 与注释 (//、/* */ ) —— 保护
*       localStorage 键名等字符串与注释中的字样
*   2. 跳过属性访问 (前一非空白字符为 '.' ) —— 保护 richOutline.activeId
*   3. 跳过对象字面量键 (后一非空白字符为 ':' ) —— 保护 { activeId: null }
*
* 用法: node tools/refactor-rename.js [--dry-run]

```

```
* --dry-run 只统计不写文件
* ===== */
'use strict';
const fs = require('fs');
const path = require('path');
const ROOT = path.join(__dirname, '..');
const PARTS_DIR = path.join(ROOT, 'src-app');
const MAPPING = {
  docs: 'state.docs',
  activeId: 'state.activeId',
  previewOn: 'state.previewOn',
  currentVisual: 'state.currentVisual'
};
const isWordStart = (c) => /[A-Za-z_$]/.test(c);
const isWord = (c) => /[A-Za-z0-9_$]/.test(c);
const isBlank = (c) => c === ' ' || c === '\t' || c === '\n' || c === '\r';
/** 在已输出字符串中找前一个非空白字符 */
function prevNonBlank(str) {
  for (let k = str.length - 1; k >= 0; k--) {
    if (!isBlank(str[k])) return str[k];
  }
  return '';
}
/** 词法安全替换: 返回 { out, replaced } */
function refactor(src) {
  let out = '';
  let replaced = 0;
  const n = src.length;
  let i = 0;
  while (i < n) {
    const c = src[i];
    // 行注释
    if (c === '/' && src[i + 1] === '/') {
      let j = src.indexOf('\n', i);
      if (j < 0) j = n;
      out += src.slice(i, j); i = j;
      continue;
    }
    // 块注释
    if (c === '/' && src[i + 1] === '*') {
      const j = src.indexOf('*/', i + 2);
      const end = j < 0 ? n : j + 2;
      out += src.slice(i, end); i = end;
      continue;
    }
    // 正则字面量 (启发式): 前一非空白字符不是「操作数/字符串结尾」类字符
    // 时按正则解析, 避免正则内的 " 误入字符串模式导致后续区域漏替换
    // (如 /[\\\:?*"<>|]/g 中的双引号)。
    if (c === '/' && !/[0-9\\]\x27_$A-Za-z]/.test(prevNonBlank(out))) {
      let j = i + 1;
```

```
let inClass = false;
while (j < n) {
  const ch = src[j];
  if (ch === '\\') { j += 2; continue; }
  if (ch === '[') inClass = true;
  else if (ch === ']') inClass = false;
  else if (ch === '/' && !inClass) { j++; break; }
  j++;
}
while (j < n && /[a-z]/i.test(src[j])) j++; // flags
out += src.slice(i, j); i = j;
continue;
}
// 字符串 / 模板
if (c === '"' || c === "'" || c === '`') {
  const q = c;
  let j = i + 1;
  while (j < n) {
    if (src[j] === '\\') { j += 2; continue; }
    if (src[j] === q) { j++; break; }
    j++;
  }
  out += src.slice(i, j); i = j;
  continue;
}
// 标识符
if (isWordStart(c)) {
  let j = i + 1;
  while (j < n && isWord(src[j])) j++;
  const word = src.slice(i, j);
  if (Object.prototype.hasOwnProperty.call(MAPPING, word)) {
    // 前一非空白字符（在已输出缓冲中往前找）
    let prev = '';
    for (let k = out.length - 1; k >= 0; k--) {
      if (!isBlank(out[k])) { prev = out[k]; break; }
    }
    // 后一非空白字符
    let k2 = j;
    while (k2 < n && isBlank(src[k2])) k2++;
    const next = k2 < n ? src[k2] : '';
    if (prev !== '.' && next !== ':') {
      out += MAPPING[word];
      replaced++;
      i = j;
      continue;
    }
  }
}
out += word; i = j;
continue;
}
```

```

    out += c; i++;
  }
  return { out, replaced };
}

const dryRun = process.argv[2] === '--dry-run';
let total = 0;
for (const f of fs.readdirSync(PARTS_DIR).filter((x) => x.endsWith('.js'))) {
  const file = path.join(PARTS_DIR, f);
  const src = fs.readFileSync(file, 'utf8');
  const { out, replaced } = refactor(src);
  total += replaced;
  if (replaced > 0) {
    if (dryRun) {
      console.log('[dry-run] ' + f + ': ' + replaced + ' 处将替换');
    } else {
      fs.writeFileSync(file, out);
      console.log('[refactor] ' + f + ': ' + replaced + ' 处已替换');
    }
  }
}

console.log(dryRun ? '[dry-run] 合计 ' + total + ' 处（未写文件）' : '[refactor] 合计替换 ' + total + ' 处');
// ===== tools/split-css.js =====
/* =====
* split-css.js —— Phase 5: style.css 按行分段拆分为三层
*
* 分段（1-based, 含端点；保持原层叠顺序）：
*   base.css      L1-47   token(:root) + reset + 基础布局
*   layout.css    L49-436 侧边栏 / 主区域 / App Bar 布局容器
*   components.css L438-末 工具栏 / 菜单 / 弹窗 / 预览 / 浮层等组件
*
* 说明：纯机械行切分，不重排任何规则，层叠顺序与原文件一致。
* 用法：node tools/split-css.js
* ===== */
'use strict';
const fs = require('fs');
const path = require('path');
const ROOT = path.join(__dirname, '..');
const SRC = path.join(ROOT, 'css', 'style.css');
const OUT = path.join(ROOT, 'css');
const SEGMENTS = [
  { file: 'base.css', from: 1, to: 47, header: '/* ===== base.css —— token / reset / 基础（源：style.css L1-47） ===== */' },
  { file: 'layout.css', from: 49, to: 436, header: '/* ===== layout.css —— 页面布局容器：侧边栏 / 主区域 / App Bar（源：style.css L49-436） ===== */' },
  { file: 'components.css', from: 438, to: Infinity, header: '/* ===== components.css —— 组件：工具栏 / 菜单 / 弹窗 / 预览 / 浮层（源：style.css L438 起） ===== */' }
];
const lines = fs.readFileSync(SRC, 'utf8').split(/\r?\n/);
const total = lines.length;
for (const seg of SEGMENTS) {
  const end = seg.to === Infinity ? total : seg.to;

```

```

const slice = lines.slice(seg.from - 1, end);
const content = seg.header + '\n\n' + slice.join('\n') + '\n';
fs.writeFileSync(path.join(OUT, seg.file), content);
console.log(' [split-css] ' + seg.file + ' ' + slice.length + ' 行');
}

console.log(' [split-css] 完成 (style.css 共 ' + total + ' 行, 全部覆盖)');
// ===== tools/vite-plugin-lnote.js =====
/* =====
* vite-plugin-lnote.js —— 把 src-app 拼接流程接入 Vite
*
* 背景: 前端源码按职责拆在 src-app/*.js (同闭包片段),
* 由 tools/build-app.js 拼回 js/app.js。本插件让 Vite 在
* dev/build 时自动保证 js/app.js 是最新拼接产物:
*   - buildStart: 启动时重拼一次 (确保产物/服务内容最新)
*   - configureServer: 监听 src-app/ 变化 → 重拼 + 整页 reload
* ===== */
'use strict';
const { execSync } = require('child_process');
const fs = require('fs');
const path = require('path');
const ROOT = path.join(__dirname, '..');
const SRC_DIR = path.join(ROOT, 'src-app');
const OUT_DIR = path.join(ROOT, 'dist-web');
function rebuild() {
  execSync('node tools/build-app.js', { cwd: ROOT, stdio: 'inherit' });
}

module.exports = function lNotePlugin() {
  return {
    name: 'lnote-build',
    enforce: 'pre',
    // dev/build 启动时确保 js/app.js 由 src-app 最新拼接生成
    buildStart() {
      rebuild();
    },
    configureServer(server) {
      // 监听 src-app 变化: 重新拼接 + 整页 reload (拼接产物非模块, 用整页刷新而非 HMR)
      let timer = null;
      fs.watch(SRC_DIR, { persistent: true }, (evt, filename) => {
        if (!filename || !filename.endsWith('.js')) return;
        if (timer) clearTimeout(timer);
        timer = setTimeout(function () {
          timer = null;
          try {
            rebuild();
          } catch (e) {
            console.error(' [L.Note] rebuild failed:', e.message);
            return;
          }
          server.ws.send({ type: 'full-reload' });
        }, 120);
      });
    }
  };
}

```

```

    });
  },
  // Vite 对非 module script 只警告不拷贝 (js/app.js、vendor/* 等是传统
  // script, 必须原样进产物), 这里在构建收尾时手动复制运行时脚本目录。
  // 注: css/ 已被 Vite 合并进 assets/index-*.css, 无需拷贝。
  closeBundle() {
    for (const dir of ['js', 'vendor']) {
      fs.cpSync(path.join(ROOT, dir), path.join(OUT_DIR, dir), { recursive: true });
    }
    // Vite 会给 CSS link 加 crossorigin 属性, 在 file:// (pywebview 打包)
    // 环境下会被 CORS 拦截导致样式失效, 这里统一移除。
    const htmlPath = path.join(OUT_DIR, 'index.html');
    if (fs.existsSync(htmlPath)) {
      let html = fs.readFileSync(htmlPath, 'utf8');
      html = html.replace(/\\scrossorigin(="[^"]*" )?/gi, '');
      fs.writeFileSync(htmlPath, html);
    }
    console.log('[L. Note] 已复制 js/ vendor/ 到 dist-web/ 并移除 crossorigin');
  }
};
};

# ===== installer/LNote.iss =====
#define MyAppName "L. Note"
#define MyAppVersion "1.0.0"
#define MyAppExeName "L. Note.exe"
#define MyAppAssocProgId "L. Note. Image"

[Setup]
AppId={ {9F7E6D2A-1B3C-4E5F-8A9D-0C2E4F6A8B10}
AppName={#MyAppName}
AppVersion={#MyAppVersion}
AppVerName={#MyAppName} {#MyAppVersion}
AppPublisherURL=https://stutasliu.github.io/LNote/
DefaultDirName={localappdata} \Programs\{#MyAppName}
DefaultGroupName={#MyAppName}
DisableProgramGroupPage=yes
PrivilegesRequired=lowest
OutputDir=.\release
OutputBaseFilename={#MyAppName}-setup-v{#MyAppVersion}
SetupIconFile=.\icons\L. NOTE\icon-L. NOTE. ico
Compression=lzma2/max
SolidCompression=yes
WizardStyle=modern
ShowLanguageDialog=auto
VersionInfoVersion={#MyAppVersion}.0
CloseApplications=force
RestartApplications=yes
UninstallDisplayIcon={app} \{#MyAppExeName}
UninstallDisplayName={#MyAppName} {#MyAppVersion}
[Languages]
Name: "chinesesimp"; MessagesFile: "compiler:Languages\ChineseSimplified.isl"

```

```
Name: "english"; MessagesFile: "compiler:Default.isl"
[CustomMessages]
chinesesimp.AdditionalIcons=附加图标:
chinesesimp.CreateDesktopIcon=创建桌面快捷方式
chinesesimp.AssocGroup=文件关联:
chinesesimp.AssocTask=将 L. Note 设为 .md/.markdown 与常见图片 (.png/.jpg/.jpeg/.gif/.webp/.bmp/.svg/.ico) 的默认打开程序
chinesesimp.RunApp=运行 L. Note
chinesesimp.UninstallApp=卸载 L. Note
english.AdditionalIcons=Additional icons:
english.CreateDesktopIcon=Create a &desktop shortcut
english.AssocGroup=File associations:
english.AssocTask=Set L. Note as the default app for .md/.markdown and common images (.png/.jpg/.jpeg/.gif/.webp/.bmp/.svg/.ico)
english.RunApp=Run L. Note
english.UninstallApp=Uninstall L. Note
[Tasks]
Name: "desktopicon"; Description: "{cm:CreateDesktopIcon}"; GroupDescription: "{cm:AdditionalIcons}"; Flags: unchecked
Name: "fileassoc"; Description: "{cm:AssocTask}"; GroupDescription: "{cm:AssocGroup}"
[Files]
Source: "..\dist\{#MyAppExeName}"; DestDir: "{app}"; Flags: ignoreversion
[Registry]
Root: HKCU; Subkey: "Software\Classes\.md"; ValueType: string; ValueName: ""; ValueData: "{#MyAppAssocProgId}"; Flags:
uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.md\OpenWithProgids"; ValueType: string; ValueName: "{#MyAppAssocProgId}"; ValueData: "";
Flags: uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.markdown"; ValueType: string; ValueName: ""; ValueData: "{#MyAppAssocProgId}"; Flags:
uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.markdown\OpenWithProgids"; ValueType: string; ValueName: "{#MyAppAssocProgId}"; ValueData:
""; Flags: uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.png"; ValueType: string; ValueName: ""; ValueData: "{#MyAppAssocProgId}"; Flags:
uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.png\OpenWithProgids"; ValueType: string; ValueName: "{#MyAppAssocProgId}"; ValueData: "";
Flags: uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.jpg"; ValueType: string; ValueName: ""; ValueData: "{#MyAppAssocProgId}"; Flags:
uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.jpg\OpenWithProgids"; ValueType: string; ValueName: "{#MyAppAssocProgId}"; ValueData: "";
Flags: uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.jpeg"; ValueType: string; ValueName: ""; ValueData: "{#MyAppAssocProgId}"; Flags:
uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.jpeg\OpenWithProgids"; ValueType: string; ValueName: "{#MyAppAssocProgId}"; ValueData: "";
Flags: uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.gif"; ValueType: string; ValueName: ""; ValueData: "{#MyAppAssocProgId}"; Flags:
uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.gif\OpenWithProgids"; ValueType: string; ValueName: "{#MyAppAssocProgId}"; ValueData: "";
Flags: uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.webp"; ValueType: string; ValueName: ""; ValueData: "{#MyAppAssocProgId}"; Flags:
uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.webp\OpenWithProgids"; ValueType: string; ValueName: "{#MyAppAssocProgId}"; ValueData: "";
Flags: uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.bmp"; ValueType: string; ValueName: ""; ValueData: "{#MyAppAssocProgId}"; Flags:
uninsdeletevalue; Tasks: fileassoc
```

```

Root: HKCU; Subkey: "Software\Classes\.bmp\OpenWithProgids"; ValueType: string; ValueName: "{#MyAppAssocProgId}"; ValueData: "";
Flags: uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.svg"; ValueType: string; ValueName: ""; ValueData: "{#MyAppAssocProgId}"; Flags:
uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.svg\OpenWithProgids"; ValueType: string; ValueName: "{#MyAppAssocProgId}"; ValueData: "";
Flags: uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.ico"; ValueType: string; ValueName: ""; ValueData: "{#MyAppAssocProgId}"; Flags:
uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.ico\OpenWithProgids"; ValueType: string; ValueName: "{#MyAppAssocProgId}"; ValueData: "";
Flags: uninsdeletevalue; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.md\OpenWithProgids"; ValueType: none; Flags: uninsdeletekeyifempty; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.markdown\OpenWithProgids"; ValueType: none; Flags: uninsdeletekeyifempty; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.png\OpenWithProgids"; ValueType: none; Flags: uninsdeletekeyifempty; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.jpg\OpenWithProgids"; ValueType: none; Flags: uninsdeletekeyifempty; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.jpeg\OpenWithProgids"; ValueType: none; Flags: uninsdeletekeyifempty; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.gif\OpenWithProgids"; ValueType: none; Flags: uninsdeletekeyifempty; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.webp\OpenWithProgids"; ValueType: none; Flags: uninsdeletekeyifempty; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.bmp\OpenWithProgids"; ValueType: none; Flags: uninsdeletekeyifempty; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.svg\OpenWithProgids"; ValueType: none; Flags: uninsdeletekeyifempty; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\.ico\OpenWithProgids"; ValueType: none; Flags: uninsdeletekeyifempty; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\{#MyAppAssocProgId}"; ValueType: string; ValueName: "FriendlyAppName"; ValueData:
"{#MyAppName}"; Flags: uninsdeletekey; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\{#MyAppAssocProgId}\DefaultIcon"; ValueType: string; ValueName: ""; ValueData:
""""{app}\{#MyAppExeName}""",0"; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\{#MyAppAssocProgId}\shell\open\command"; ValueType: string; ValueName: ""; ValueData:
""""{app}\{#MyAppExeName}"" ""%1"""; Tasks: fileassoc
Root: HKCU; Subkey: "Software\Classes\Applications\{#MyAppExeName}\shell\open\command"; ValueType: string; ValueName: "";
ValueData: """"{app}\{#MyAppExeName}"" ""%1"""; Flags: uninsdeletekey; Tasks: fileassoc
[Icons]
Name: "{autoprograms}\{#MyAppName}\{#MyAppName}"; Filename: "{app}\{#MyAppExeName}"; WorkingDir: "{app}"
Name: "{autoprograms}\{#MyAppName}\{cm:UninstallApp}"; Filename: "{uninstallexe}"
Name: "{autodesktop}\{#MyAppName}"; Filename: "{app}\{#MyAppExeName}"; WorkingDir: "{app}"; Tasks: desktopicon
[Run]
Filename: "{app}\{#MyAppExeName}"; Description: "{cm:RunApp}"; WorkingDir: "{app}"; Flags: nowait postinstall; Check: not
IsGuardDrivenUpdate
[Code]
function IsGuardDrivenUpdate(): Boolean;
begin
    // 更新守护进程 (v0.21.17+ 自动更新) 启动的静默安装带 LNOTE_UPDATE_GUARD=1
    // 标记: 跳过 [Run], 改由守护进程在安装器完全退出后拉起新版本, 避免双重启动
    // 与安装器收尾重叠导致 PyInstaller onefile 解压竞态 (Failed to load Python
    // DLL 报错)。手动交互安装 / 旧版本自动更新无此标记, 仍由 [Run] 启动。
    Result := GetEnv('LNOTE_UPDATE_GUARD') = '1';
end;
# ===== LNote.spec =====
# -*- mode: python ; coding: utf-8 -*-
a = Analysis(
    ['main.py'],
    pathex=[],
    binaries=[],

```

```

# 打包资源（Phase 3）：主窗口加载源根 app.html（编辑器壳，原样打包）；
# js/ vendor/ 取自 Vite 产物 dist-web/（由插件从源根拷贝）。
# 独立窗口页 compare.html / image_viewer.html 未走 Vite，原样从源根打包。
# 注意：打包前先执行 `npm run build` 生成 dist-web/。
datas=[
    ('app.html', '.'),
    ('compare.html', '.'),
    ('image_viewer.html', '.'),
    ('dist-web/assets', 'assets'),
    ('dist-web/js', 'js'),
    ('dist-web/vendor', 'vendor'),
    ('css', 'css'),
    ('icons', 'icons'),
],
hiddenimports=[],
hookspath=[],
hooksconfig={},
runtime_hooks=[],
excludes=[],
noarchive=False,
optimize=0,
)
pyz = PYZ(a.pure)
exe = EXE(
    pyz,
    a.scripts,
    a.binaries,
    a.datas,
    [],
    name='L.Note',
    icon='icons/L.NOTE/icon-L.NOTE.ico',
    debug=False,
    bootloader_ignore_signals=False,
    strip=False,
    upx=True,
    upx_exclude=[],
    runtime_tmpdir=None,
    console=False,
    disable_windowed_traceback=False,
    argv_emulation=False,
    target_arch=None,
    codesign_identity=None,
    entitlements_file=None,
)
# ===== LNoteMac.spec =====
# -*- mode: python ; coding: utf-8 -*-
# macOS 打包配置：生成 L.Note.app
# 前置：前端产物 dist-web/ 已生成（Windows 上 npm run build 后拷贝整个仓库到 Mac，
#       或 Mac 上自行 npm install && npm run build）。
# 用法：python3 -m PyInstaller --clean -y LNoteMac.spec

```

```
# 说明: target_arch='universal2' 需使用官方 universal2 版 Python;
#      仅打单架构时删除该行 (继承当前 Python 架构)。
a = Analysis(
    ['main.py'],
    pathex=[],
    binaries=[],
    datas=[
        ('dist-web/index.html', '.'),
        ('compare.html', '.'),
        ('image_viewer.html', '.'),
        ('dist-web/assets', 'assets'),
        ('dist-web/js', 'js'),
        ('dist-web/vendor', 'vendor'),
        ('css', 'css'),
        ('icons', 'icons'),
    ],
    hiddenimports=[],
    hookspath=[],
    hooksconfig={},
    runtime_hooks=[],
    excludes=[],
    noarchive=False,
    optimize=0,
)
pyz = PYZ(a.pure)
exe = EXE(
    pyz,
    a.scripts,
    [],
    exclude_binaries=True,
    name='L.Note',
    debug=False,
    bootloader_ignore_signals=False,
    strip=False,
    upx=False,          # macOS 不推荐 upx
    console=False,      # GUI 应用, 无终端
    target_arch='universal2',
)
coll = COLLECT(exe, a.binaries, a.datas, name='L.Note')
app = BUNDLE(
    coll,
    name='L.Note.app',
    icon='icons/L.NOTE/icon-L.NOTE.icns',
    bundle_identifier='com.xiansheng.lnote',
    info_plist={
        'CFBundleDisplayName': 'L.Note',
        'CFBundleShortVersionString': '1.0.0',
        'CFBundleVersion': '1.0.0',
        'LSMinimumSystemVersion': '11.0',
        'NSHighResolutionCapable': True,
```

```

        'NSRequiresAquaSystemAppearance': False,
    },
)
# ===== LNoteDebug.spec =====
# 诊断用 spec: console=True 捕获报错, 构建为 L.NoteDebug.exe
a = Analysis(
    ['main.py'],
    pathex=[],
    binaries=[],
    datas=[
        ('app.html', '.'),
        ('compare.html', '.'),
        ('image_viewer.html', '.'),
        ('dist-web/assets', 'assets'),
        ('dist-web/js', 'js'),
        ('dist-web/vendor', 'vendor'),
        ('css', 'css'),
        ('icons', 'icons'),
    ],
    hiddenimports=[],
    hookspath=[],
    hooksconfig={},
    runtime_hooks=[],
    excludes=[],
    noarchive=False,
    optimize=0,
)
pyz = PYZ(a.pure)
exe = EXE(
    pyz,
    a.scripts,
    a.binaries,
    a.datas,
    [],
    name='L. NoteDebug',
    icon='icons/L. NOTE/icon-L. NOTE.ico',
    debug=False,
    bootloader_ignore_signals=False,
    strip=False,
    upx=True,
    upx_exclude=[],
    runtime_tmpdir=None,
    console=True,
    disable_windowed_traceback=False,
    argv_emulation=False,
    target_arch=None,
    codesign_identity=None,
    entitlements_file=None,
)
# ===== register-file-assoc.ps1 =====

```

```

# Register L.Note as an image + markdown handler:
#   - "Open with" entry (right-click -> Open with -> L.Note)
#   - default double-click handler for common image types and .md/.markdown
# Writes to HKCU only (no admin rights required).
#
# Run it either by:
#   1) double-clicking register-file-assoc.bat, or
#   2) powershell -NoProfile -ExecutionPolicy Bypass -File register-file-assoc.ps1
#
# After running, double-click a .md file (or right-click -> "Open with" -> L.Note).
$errorActionPreference = 'Stop'
# Locate the built executable relative to this script (source stays ASCII-safe).
$exe = Join-Path $PSScriptRoot 'dist\L.Note.exe'
if (-not (Test-Path -LiteralPath $exe)) {
    Write-Host "[ERROR] EXE not found: $exe" -ForegroundColor Red
    Write-Host "Build it first: python -m PyInstaller LNote.spec --noconfirm" -ForegroundColor Yellow
    exit 1
}
$progId = 'L.Note.Image'
$command = '"' + $exe + '" "%1"'
$exts = '.png', '.jpg', '.jpeg', '.gif', '.webp', '.bmp', '.svg', '.ico'
$docExts = '.md', '.markdown'
# 1) Register the app in the "Open with" dialog (per-user Applications key).
$appCmd = "HKCU:\Software\Classes\Applications\L.Note.exe\shell\open\command"
New-Item -Path $appCmd -Force | Out-Null
Set-Item -Path $appCmd -Value $command
# 2) Define a ProgID pointing to the same command (friendly name + icon).
$prog = "HKCU:\Software\Classes\$progId"
New-Item -Path "$prog\shell\open\command" -Force | Out-Null
Set-Item -Path "$prog\shell\open\command" -Value $command
New-Item -Path "$prog\DefaultIcon" -Force | Out-Null
Set-Item -Path "$prog\DefaultIcon" -Value "$exe,0"
New-ItemProperty -Path $prog -Name 'FriendlyAppName' -Value 'L.Note' -PropertyType String -Force | Out-Null
# 3) Attach the ProgID to common image extensions + make it the default handler.
foreach ($e in $exts) {
    # Open-with entry
    $oid = "HKCU:\Software\Classes\$e\OpenWithProgid"
    New-Item -Path $oid -Force | Out-Null
    New-ItemProperty -Path $oid -Name $progId -Value '' -PropertyType String -Force | Out-Null
    # Default double-click handler
    $extKey = "HKCU:\Software\Classes\$e"
    if (-not (Test-Path $extKey)) { New-Item -Path $extKey -Force | Out-Null }
    Set-Item -Path $extKey -Value $progId
}
# 4) Attach the ProgID to markdown extensions + make it the default handler.
foreach ($e in $docExts) {
    # Open-with entry
    $oid = "HKCU:\Software\Classes\$e\OpenWithProgid"
    New-Item -Path $oid -Force | Out-Null
    New-ItemProperty -Path $oid -Name $progId -Value '' -PropertyType String -Force | Out-Null

```

```
# Default double-click handler
$extKey = "HKCU:\Software\Classes\$.e"
if (-not (Test-Path $extKey)) { New-Item -Path $extKey -Force | Out-Null }
Set-Item -Path $extKey -Value $progId
}

Write-Host '[OK] L.Note registered as the default image + markdown handler.' -ForegroundColor Green
REM ===== register-file-assoc.bat =====

@echo off

powershell -NoProfile -ExecutionPolicy Bypass -File "%dp0register-file-assoc.ps1"

pause

# ===== build_mac.sh =====

#!/bin/bash

# L.Note macOS 一键构建脚本

# 用法: bash build_mac.sh [skip-deps] [skip-sign]
# skip-deps: 跳过 pip 安装依赖 (已装过时用)
# skip-sign: 跳过签名与公证 (未配置证书时用)
# 注意: 必须在一台 macOS 机器上执行; 需要 Apple Developer 证书时才签名。

set -e

cd "$(dirname "$0")"

if [ "$1" != "skip-deps" ]; then
    echo "==> 安装依赖"
    python3 -m pip install --upgrade pip
    python3 -m pip install pyinstaller pywebview pyobjc-framework-Cocoa pyobjc-framework-WebKit pillow
fi

echo "==> 生成 .icns 图标"
python3 tools/gen_icns.py

echo "==> PyInstaller 打包 .app"
python3 -m PyInstaller --clean -y LNoteMac.spec
APP="dist/L.Note.app"

[ -d "$APP" ] || { echo "打包失败: 未找到 $APP"; exit 1; }

if [ "$1" != "skip-sign" ] && [ "$2" != "skip-sign" ]; then
    # 若已配置 Developer ID 证书, 则签名 + 公证 + 盖章
    if security find-identity -v -p codesigning 2>/dev/null | grep -q "Developer ID Application"; then
        echo "==> 代码签名"
        codesign --force --deep --sign "Developer ID Application" --options runtime "$APP"
        echo "==> 公证 (notarytool)"
        if [ -n "$APPLE_ID" ] && [ -n "$TEAM_ID" ] && [ -n "$APPLE_PASSWORD" ]; then
            xcrun notarytool submit "$APP" --apple-id "$APPLE_ID" --team-id "$TEAM_ID" --password "$APPLE_PASSWORD" --wait
            xcrun stapler staple "$APP"
            echo "==> 公证完成"
        else
            echo "==> 跳过公证: 未设置 APPLE_ID / TEAM_ID / APPLE_PASSWORD 环境变量"
        fi
    else
        echo "==> 跳过签名: 本机无 Developer ID Application 证书"
    fi
fi

echo ""

echo "==> 完成: $APP"

echo "==> 分发时可用: hdiutil create -volname 'L.Note' -srcfolder $APP -ov -format UDZO dist/L.Note-0.20.44-macos.dmg"
```